

Széchenyi István Egyetem
Gépészmérnöki, Informatikai és Villamosmérnöki Kar
Informatika Tanszék

Fotóalbum szelektáló alkalmazás fejlesztése

Hollósi János
Németh Norbert

Török Kristóf
Mérnök informatikus

Győr
2025



**SZÉCHENYI
EGYETEM**
UNIVERSITY OF GYŐR



**INFORMATIKA
TANSZÉK**
DEPARTMENT OF COMPUTER SCIENCE

SZAKDOLGOZAT

Fotóalbum szelektáló alkalmazás fejlesztése

Török Kristóf

Mérnökinformatikus BSc szak

2025

ADATLAP SZAKDOLGOZAT TÉMA ENGEDÉLYEZÉSÉHEZ

Hallgató adatai

Név: Török Kristóf
Cím: 9762 Tanakajd, Petőfi Sándor utca 35/D
Telefon: +36204371573
Szak: Mérnök informatikus
Képzési szint: BSc
Tagozat: nappali

Neptun-kód: UWQK95

e-mail: torokkristof1998@gmail.com

A szakdolgozat adatai

Kezdő tanév és félév: 2020/21 Tavaszi
Várható leadás: 2021/22 Ősz
Cím: Fotóalbum szelektáló alkalmazás fejlesztése
Nyelv: magyar
Típus: nyilvános
Rövid leírás, részfeladatok:

Célom egy olyan szoftver megvalósítása, amely alkalmas lehet fotósorozatok automatizált feldolgozására. Amely képes felismerni ugyanazon látványvilágról készült több képet és ezek közül kiválasztani a legjobb felvételeket. Ha lehetséges, több kép felhasználásával állítson elő jobb felvételt. Ismerje fel a különböző képalkotási hibákat (elmosódás, túlexponálás), amennyiben lehetséges javítsa ezeket minőségén, illetve ahol a képminőség egy meghatározott szint alatti azokat javasolja törlésre.

A szoftver elkészítését előzi meg a tervezés. Itt kerül sor a fejlesztőkörnyezet, programnyelv, különböző gépi látás csomagok kiválasztására és a programrészek meghatározására. Ezután kerülhet sor a szoftver elkészítésére és a programrészek implementálására. Ha a szoftver elkészült kerül sor a tesztelésre, azon belül a tesztkészletek kiválasztására vagy előállítására, majd a tesztelésre és a tesztelés eredményeinek kiértékelésére. A szoftverhez dokumentáció is készül, ami tartalmaz minden információt a programmal kapcsolatban.

Belső konzulens adatai

Név: Hollósi János
Tanszék: Informatika Tanszék
Beosztás: Egyetemi tanársegéd

Külső konzulens adatai

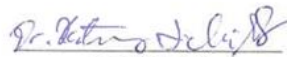
Név: Németh Norbert
Munkahely: TDK Hungary Components Kft.
Beosztás: Fejlesztőmérnök
Cím: 9700 Szombathely, Csaba utca 30.
Telefonszám: +36205798070

e-mail: norbert.nemeth@tdk-electronics.tdk.com

Győr, 2021. január 22.


Belső konzulens


Külső konzulens


Tanszékvezető

Nyilatkozat

Alulírott, Török Kristóf (UWQK95), mérnök informatikus, BSc szakos hallgató kijelentem, hogy a Fotóalbum szelektáló alkalmazás fejlesztése című szakdolgozat feladat kidolgozása a saját munkám, abban csak a megjelölt forrásokat, és a megjelölt mértékben használtam fel, az idézés szabályainak megfelelően, a hivatkozások pontos megjelölésével.

Eredményeim saját munkán, számításokon, kutatáson, valós méréseken alapulnak, és a legjobb tudásom szerint hitelesek.

Győr, 2025.05.15.


hallgató

Kivonat

A szakdolgozat célja egy olyan szoftver fejlesztése, amely alkalmas fotósorozatok feldolgozására. A szoftver képes felismerni ugyanazon a látványvilágról készült képeket és ezek közül kiválasztani a legjobb felvételeket. Felismeri a különböző képalkotási hibákat (elmosódás, túl- és alulexponálás, piros szem effektus és lila rojtosodás), amennyiben lehetséges javítson a detektált hibák minőségén, illetve ahol a képminőség egy meghatározott szint alatti azokat javasolja törlésre. Végül a már javított képeket a rajtuk található látványvilág alapján egy létrehozott mappastruktúrába menti el.

A dokumentum első felében a képalkotási hibákról, valamint a szoftver elkészítéséhez szükséges szoftverfejlesztési eszközökről lesz szó. Minden fejlesztési eszköz leírásában kifejttem, hogy hogyan működnek, mire használhatók és miért szükségesek a szoftver elkészítéséhez.

A második felében az elkészült szoftver fejlesztésének folyamatáról, valamint az egyes részfeladatok megoldásáról lesz szó, a képek betöltése, előfeldolgozása, a képeken található hibák detektálása és javítása, valamint a javított képek osztályozása. A legvégén a program tesztelésének eredménye van kifejtve. A folyamatok könnyebb megértéséhez szemléltető ábrák és a programban használt kódrészletek segítik a leírások könnyebb megértését.

Abstract

The aim of this thesis is to develop software capable of processing photo series. The software can recognize images depicting the same scene and select the best shots among them. It detects various image quality issues (such as blur, overexposure and underexposure, red-eye effect, and purple fringing), improves the quality of the detected issues where possible, and recommends deletion for images falling below a certain quality threshold. Finally, the corrected images are saved into a generated folder structure based on their visual content.

The first part of the document discusses the image quality issues and the software development tools required for building the application. Each development tool is described in terms of how it works, what it can be used for, and why it is necessary for the implementation of the software.

The second part covers the development process of the completed software and the solutions for the individual subtasks, including image loading, preprocessing, error detection and correction, and the classification of the corrected images. The final section presents the results of the program testing. To aid understanding, the document includes illustrative diagrams and code snippets from the application.

Tartalomjegyzék

1 Bevezetés.....	1
2 Elméleti háttér.....	2
2.1 Képképzési hibák	2
2.1.1 Exponálás	2
2.1.2 Elmosódások.....	3
2.1.3 Purple Fringing Aberration	5
2.2 Probléma bemutatása	6
2.3 Digitális képek hisztogramja	8
2.4 Klasszikus hisztogram kiegyenlítés	9
2.5 Kontrasztkorlátozott adaptív hisztogram kiegyenlítés (CLAHE)	10
2.6 Morfológiai műveletek	10
2.7 Laplacian operátor	13
2.8 Diszkrét Fourier transzformáció	14
2.9 Gamma korrekció	15
2.10 Point Spread Function	16
2.11 Wiener dekonvolúció.....	17
2.12 Gépi tanulás	18
2.12.1 Megközelítései.....	19
2.13 Neurális hálózatok	19
2.13.1 Tanítás.....	20
2.13.2 Előreecsatolt neurális hálózatok	20
2.13.3 Visszacsatolt neurális hálózatok	21
2.14 Konvolúciós neurális hálózatok	22
2.15 Hasonló munkák	23
2.15.1 Túlexponálás korrigálása és detektálása.....	23
2.15.2 Elmosódások korrigálása és detektálása.....	23
2.15.3 Piros szem detektálása és korrigálása.....	24
2.15.4 Purple Fringe detektálása és korrigálása	24
2.15.5 Képosztályozás	24
2.16 Fejlesztő környezet, programnyelv, programcsomagok	25
3 Megvalósítás.....	27
3.1 Program használatának előfeltételei	27
3.2 Segédosztályok konstansok	27
3.2.1 Fixen definiált konstans paraméterek	28
3.2.2 Dinamikusan számolt paraméterek.....	28

3.2.3	Segédosztályok	29
3.3	Grafikus felület.....	29
3.4	Képek betöltése és előfeldolgozása	31
3.5	Mappaválasztás és képek betöltése:	31
3.6	Előfeldolgozási lépések	31
3.7	Képalkotási hibák detektálása	32
3.7.1	Expozíció	32
3.7.2	Elmosódás.....	33
3.7.3	Piros szem.....	35
3.7.4	Purple Fringe	36
3.8	Képalkotási hibák javítása	39
3.8.1	Elmosódottság javítása	40
3.8.2	Expozíciós hibák javítása	42
3.8.3	Piros szem hibák javítása.....	45
3.8.4	Purple fringe hiba javítása	46
3.9	Osztályozás	49
3.9.1	Automatikus osztályozás YOLOv8 modellel	49
4	Tesztelés.....	50
4.1	Módszer	50
4.2	Tesztkészlet.....	50
4.3	Eredmények, levont következtetések	51
4.3.1	Előfeldolgozás	51
4.3.2	Detektálások	51
4.3.3	Javítások	52
4.3.4	Osztályozás	52
5	Összegzés	56
6	Irodalomjegyzék	57
7	Mellékletek	60

1 Bevezetés

A fotózás az elmúlt években egyre nagyobb népszerűsége tett szert. A digitális fényképezőgépek és az egyre jobb kamerákkal felszerelt okostelefonok széles körű megjelenése és sokoldalúsága vezetett ahhoz, hogy az emberek szeretnék életük minden emlékezetes pillanatát fotó formájában megörökíteni. Születésnap, nyaralás vagy más esemény megörökítése érdekében az emberek gyakran készítenek fotó sorozatokat, abban a reményben, hogy néhány jó fotót, amely megfelel az igényeknek, később kiválogathatnak szerkesztésre, közzétételre vagy megosztásra. A fotó sorozatok megjelenését tovább fokozzák az olyan megoldások, mint például az okostelefonokon elérhető sorozatfelvétel mód, amelynek segítségével a felhasználó több szinte azonos vagy hasonló képet készíthet. Ezen megoldásoknak köszönhetően a felhasználók rendkívül hatalmas fotógyűjteményekhez juthatnak, ahol ugyanazon pillanatról vagy látványvilágról több kép áll rendelkezésre. A hatalmas fotógyűjtemények szelektálása és kezelése, fárasztó és időigényes folyamat. Nehéz döntések sorozatát hozza magával, mint például, hogy melyik képet töröljük és melyiket tartjuk meg.

Jelenleg sok alkalmazás, mint például az iPhoto és a Picasa, lehetővé teszi a felhasználó számára, hogy rendszerezze a fotó sorozatokat idő vagy földrajzi információk alapján. Ezzel szemben sok online fotómegosztó közösségi oldal, például Facebook és a Google Fotók arcokat és más magasabb szintű tartalmakat vonnak ki címkézési célokra. Ezek az eszközök javítják az albumok elrendezését, viszont egyikük sem nyújt olyan lehetőséget a felhasználók számára, amely segítséget ad a rossz képek automatikus kiválogatására vagy a hasonló képeket tartalmazó gyűjtemények között a legjobb képek megtalálására. Jelenleg a felhasználóknak még mindig saját maguknak kell foglalkozniuk ezzel a problémával.

2 Elméleti háttér

2.1 Képkészítési hibák

Számos oka lehet annak, hogy egy fénykép rossz minőségű. Most áttekintem melyek azok a képkészítési hibák, amelyek miatt a képek sokszor nem olyan minőségűek, amilyeneket szerettünk volna készíteni. A leggyakoribb képkészítési hibák közé tartozik a rossz megvilágítás és a különféle elmosódások. [1]

A rossz megvilágítás olyan képekre vonatkozik, amelyeket a megfelelő fény elérése nélkül készítettek, Ez azt eredményezheti, hogy a képek tompának vagy sötétnek tűnnek. A rossz megvilágítással rendelkező képeket másnéven túlexponálnak vagy alul exponálnak is nevezzük. [1]

Elmosódások nagyon gyakoriak és különféle tényezők miatt keletkezhetnek fényképezés során. Ezek a tényezők lehetnek például a tárgy mozgása, a kamera remegése vagy a fényképezőgép lencséje nincs fókuszban. [2]

A rossz minőségű fényképeken gyakran találkozhatunk még Purple Fringing Aberration jelenséggel, amelyek általában az alacsony felszereltségű kamerák által készített fényképeken fordulnak elő. Ez az optikai torzulás általában a széles spektrumú megvilágítás fényes területei mellett lévő sötét szélek színeződéseként és világosításaként látható, mint például a nappali fény vagy a különböző típusú gázkisüléses lámpák. [11]

2.1.1 Exponálás

Az exponálás szabályozza, hogy mennyi fény juthat a kamera érzékelőjére. Ebből adódóan a fénykép világosságát a megjelenített fény mennyisége határozza meg. A fotózásban, a „dinamikus tartomány” – „dynamic range” kifejezést használják a mérhető legsötétebb és legfényesebb fényintenzitások közötti arányának leírására. [3] [4]

Az átlagos fényképezőgépek dinamikus tartománya nagyon korlátozott, általában 100:1-hez, ami sokkal kevesebb mint a való életben. A nagy kontraszttal rendelkező helyszínek, mint például napfény alatti kültéri környezet, nagyon magas dinamikus tartománnyal rendelkezhetnek, 105-től 109-ig. Az ilyen helyszínek esetében nagyon nehéz minden területet jól megvilágítani. Emiatt a túlexponálás keletkezik. Túlexponálásnak nevezzük, ha a fénykép egyes világos részein a kiemelt részletek elvesznek. Ez a jelenség akkor fordul elő, amikor a kamera érzékelőjére hulló fény meghaladja azt, amit az érzékelő képes rögzíteni. Túlexponálás történik nagyon gyakran a mindennapi fotózásban a helyszínek nagy dinamika tartománya

(HDR) miatt. [3]

Előfordulhat, hogy a fényképeken fényes területek helyett sötét területekkel találkozunk, amelyek szinte megkülönböztethetetlenek a fekete színtől. Ennek oka, hogy nem jut elég fény a fényképezőgép érzékelőjére. Ezeket a területeket alulexponáltnak nevezzük. A következő ábrán (1. ábra) két fényképet láthatunk, amelyeken túlexponált és alulexponált területek vannak:



1. ábra: Túlexponált (bal) és alulexponált (jobb) kép [4]

Gyakorlatban a fotósok néha megpróbálják csökkenteni az exponálási értéket azért, hogy megakadályozzák a túlexponálást. Azonban az exponálási érték túlzott csökkentése elhomályosítja a fényképet. [3]

Túlexponálás javítása vagy elkerülése többféle módon lehetséges. Néhány nagy dinamika tartományú képrögzítéssel foglalkozó munkában, a teljes dinamikus tartomány rögzítését célozza meg. Tónustérképezési technikákkal az alacsony dinamika tartományú képeket az alacsony dinamika tartományú (LDR) képekhez hozzárendelik, elkerülve ezzel a túlexponálást. Ez azonban nagyon drága megoldás, a HDR kamerák magas árai miatt. Drága kamerák helyett vannak más létező HDR megoldások is, viszont általában ezek több felvételt igényelnek, különböző exponálási értékekkel. Az ilyen többszörös bemeneti módszerek korlátozóak, mert megkövetelik, hogy a helyszín statikus legyen. Ezenkívül a HDR rögzítés csak új fényképek esetében működik. Nem tudja kijavítani a túlexponálást a már meglévő fényképeken. [3]

2.1.2 Elmosódások

Elmosódásokkal gyakran találkozhatunk a fényképeken, amelyeket a mozgás, kézremegés vagy a helytelen fókuszálás okoz. Azonban ezek az elmosódások lehetnek szándékosak azért, hogy a fotós a fotó kifejezőerejét erősítse ezáltal. Ilyen esetekben az elmosódás nem csökkenti a kép minőségét, azonban ez az esetek kisebbik halmaza. [5]

A leggyakoribb elmosódások közé tartozik a mozgási elmosódás (motion blur) (2. és 3. ábra). Egy expozíciós perióduson belül a kamera vagy tárgyak mozgása elmosódott képeket eredményez, mivel a megvilágítás változásai az idő múlásával integrálódnak és az élesség elmosódik. Másfelől viszont a mozgás miatt elmosódott kép megtartja a mozgásról szóló információt, amely paraméterezi az elmosódást. Így támpontokat ad ahhoz, hogy a mozgást visszanyerjük egyetlen képből. [6]



2. ábra: Példák térváltozó mozgási elmosódásra [6]

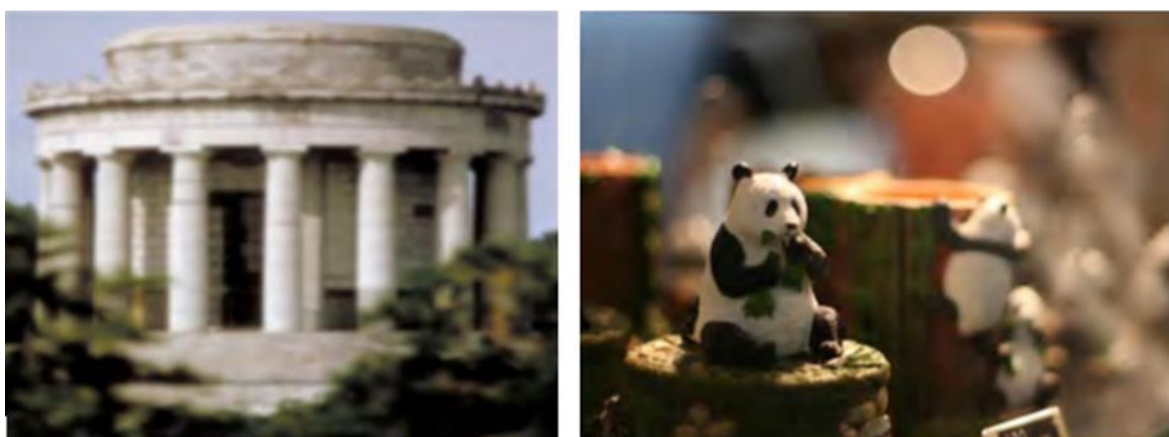
A mozgási elmosódást a pontterjedési függvény (Point Spread Function – PSF) jellemzi, amelynek paraméterei szorosan kapcsolódnak a mozgáshoz. A legegyszerűbb mozgási elmosódás a térinvariáns mozgási elmosódás. Ezt a fajta mozgást széles körben tanulmányozták. A gyakorlatban azonban, mivel a mozgás meglehetősen összetett lehet, a mozgásfoltok sokkal bonyolultabbak lehetnek ennél az egyszerű esetnél. Például az elmosódás lehet térváltozó, nemlineáris, lokális és többszörös. Több tanulmány is foglalkozik ezekkel az esetekkel, azonban a jelenlegi megoldások közül, azok a legjobbak, amelyek több bemeneti képre, felhasználói interakciókra vagy extra feltételezésekre támaszkodnak. [6]

A kamerarendszerek helytelen fókuszálása miatt létrejövő képi elmosódások (4. ábra) elsősorban a geometriai képalkotás bizonyos problémájából és a kameraobjektívek véges mélységélességéből adódnak. Ezek a képelmosódások, amelyek a fókuszálási hiba vagy a nem megfelelő fókuszálással rendelkező képalkotó lencserendszer miatt következnek be, komoly képromlást okoznak. [7]



3. ábra: Példák lokális mozgási elmosódásra [7]

Ezen elmosódások kijavítására számos képfeldolgozási technikát fejlesztettek ki. Ezen megoldások többsége a képfelvételi rendszerhez kapcsolódó pontterjedési függvényt (Point-Spread Function - PSF) becsüli, amely a kép helyreállítása érdekében a becsült elmosódási függvény alapján térinvariáns dekonvolúciót végez. [7]



4. ábra: Teljesen elmosódott (bal) és részben defókuszált (jobb) kép [8]

2.1.3 Purple Fringing Aberration

Purple Fringing Aberration (5. ábra) egy gyakran előforduló színeltérés a digitális fényképezőgépek által készített képeken, amely lila fátyolként jelenik meg az objektumok körül. A lila rojtos régió elveszíti az eredeti színét és a terület torzított színnel jelenik meg. Mivel napjainkban egyre kisebbek a digitális fényképezőgépek, amelyek támogatják a kiváló minőségű kimenetet, a lila rojtosodás komoly problémává vált. [9]

Általában a lila rojtosodás a kromatikus aberrációnak tulajdonítható, azonban ez nem minden esetben igaz. A fénypontok körüli hasonló rojtosodás a lencse becsillanásából is adódhat. A fénypontok vagy sötét területek körüli színes rojtosodás oka lehet, hogy a különböző színek receptorainak dinamikatarományja vagy érzékenysége eltérő. A rojtosodás másik oka a

kromatikus aberráció a CCD pixelenként több fény összegyűjtésére használt nagyon kicsi mikrolencsékben. Mivel ezek a lencsék a zöld fény helyes fókuszálására vannak beállítva, a vörös és kék fény helytelen fókuszálása lila rojtosodást eredményez a fénypontok körül. Ez a probléma egyenletes az egész képkockán és nagyobb problémát jelent a nagyon kis pixelosztású CCD-knél. [9]



5. ábra: Példaképek Purple Fringing Aberration

A lila rojtosodás kezelésére számos módszert mutattak be. Hardveres megközelítésként a kromatikus aberráció kiküszöbölésére optimális lencseterveket javasoltak. Ezek azonban a kromatikus aberráció eltávolítására koncentrálnak és a lencsék tervezése magas költségekkel jár. A lila rojtos képek utólagos feldolgozására digitális képfeldolgozási módszereket is javasolnak. Amelyek általában két lépésből állnak. A lila területek detektálásából és az azt követő korrekciójából, amelyekben a korrekciós lépések korlátozott teljesítményt mutattak. [9]

2.2 *Probléma bemutatása*

A fotózás az elmúlt években egyre népszerűbb lett. Az emberek szeretnék életük minden emlékezetes pillanatát fotók formájában dokumentálni. Korábban a fényképezőgépek és a tárolási kapacitások elérhetősége miatt elképzelhetetlen volt, viszont napjainkban gyakran készítenek több képet ugyanazon látványvilágról vagy pillanatról. Ezt tovább fokozzák az olyan megoldások, mint például több okostelefonon elérhető sorozatfelvétel mód, amely több tucat szinte azonos képet készítenek annak érdekében, hogy minél jobb minőségű képet állítsanak

elő az adott pillanatról. Ennek eredményeként a felhasználók végül rendkívül nagy fotó gyűjteményhez juthatnak, ahol minden pillanatról vagy látványvilágról több kép áll rendelkezésre. Emiatt fotósok és egyszerű felhasználók sok időt töltenek el azzal, hogy kiválasszák a legjobb képet a sok hasonló közül. Ez a folyamat gyakran nehézkes és időigényes. [10]

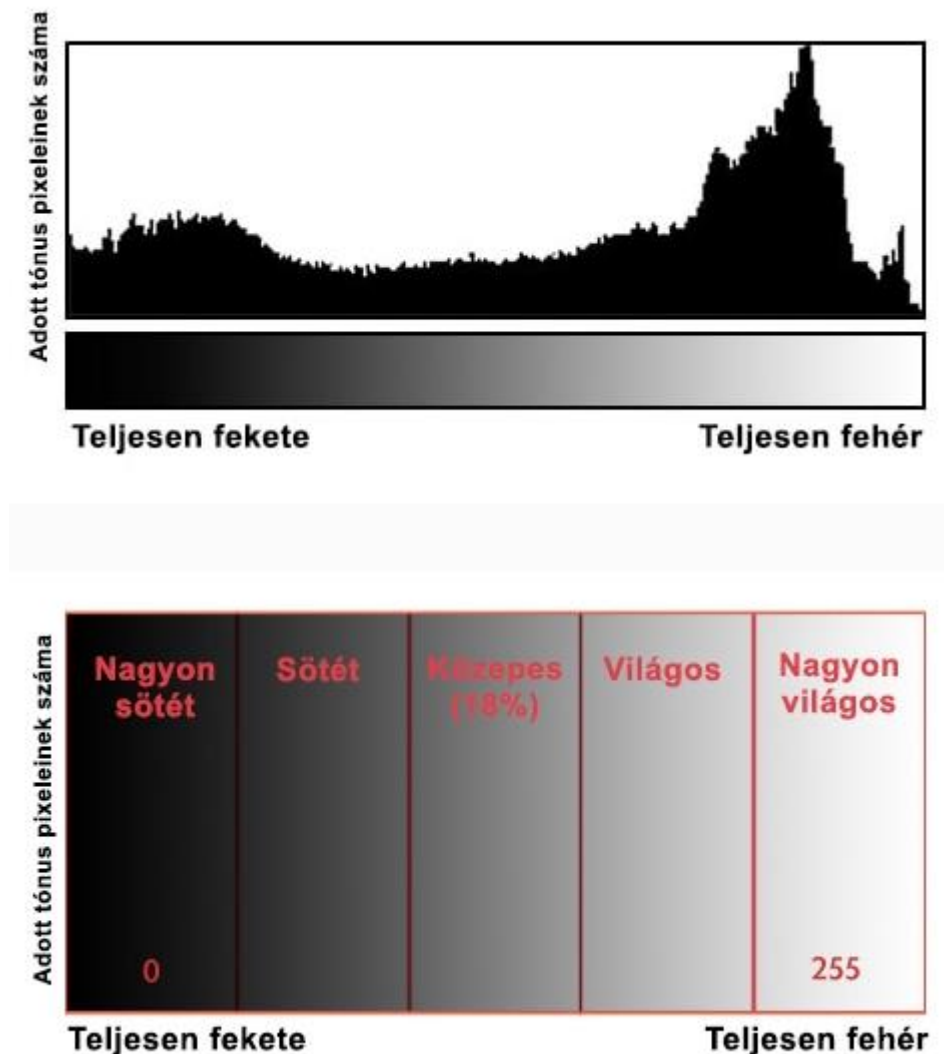
Manapság egyre több kutatási erőfeszítés történik a képminőség esztétikai szempontból történő értékeléséről. Ezek a módszerek irányíthatják a képek kiválasztását, azonban ezeket alig használják fotósorozatokra, amelyek teljesen ugyanazt a látványvilágot ábrázolják. Általában ezek változatos tartalmú képek egy nagy általános korpuszára vannak kiképezve és szoros értékelést adnak hasonló képekhez. Ennek ellenére a vizuális különbségek a fotósorozatok között nagyon finomak, emiatt a létező módszerek a képek esztétikai értékelésére, kevésbé képesek fotósorozatok kiválasztására. [10]

Máig több módszert javasoltak fotósorozatok kiválasztásának megkönnyítésére. Kuzovkin és társai [11] [12] meghatározták az egyes fotók többszintű összefüggéseit hierarchikus csoportosítással. Chang és társai [13] összegyűjtötték az első nyilvános adatkészletet, amely személyes fotóalbumok fotósorozatait tartalmazza és bemutatott egy végpontok közötti mélytanulás módszert Siamase hálózattal. Az előbb említett tanulmányokban elért kezdeti áttörés ellenére továbbra is nagy kihívást jelent a megkülönböztető jellemzők megállapítása a finom különbségek azonosítása érdekében a fotósorozatok képei között. [10]

Ezért a céloom egy olyan szoftver megvalósítása, amely alkalmas lehet fotósorozatok automatizált feldolgozására. Amely képes felismerni ugyanazon látványvilágról készült több képet és ezek közül kiválasztani a legjobb felvételeket. Amennyiben szükség és lehetőség van rá akkor több kép felhasználásával állítson elő jobb felvételt. Felismeri a képalkotási hibákat (elmosódás, túlexponálás) és ha lehetséges javít ezek minőségén, illetve ahol a képminőség egy meghatározott szint alatti akkor javaslatot tesz a felhasználó számára, hogy törölje a képet. [10]

2.3 Digitális képek hisztogramja

A kép hisztogramja, más hisztogramokhoz hasonlóan, szintén a gyakoriságot mutatja. A képhisztogram azonban a pixelek intenzitásértékeinek gyakoriságát mutatja. A képhisztogramban megfigyelhető a szürkeségi intenzitások mértéke és ezen intenzitások gyakorisága, amely a következő ábrán figyelhető meg (6. ábra). [14]



6. ábra: Egy kép hisztogramja [19]

A hisztogram vízszintes tengelye a pixel értékek tartományát mutatja. Mivel a kép 8 bpp-es (bits per pixel), ez azt jelenti, hogy 256 szürkeárnyalatot tartalmaz. Ezért a vízszintes tengely tartománya 0-tól indul és 255-nél ér véget. Míg az függőleges tengelyen ezeknek az intenzitásoknak a száma látható. [14]

Ha a hisztogramban néhány képpont a minimum vagy maximum szint közelében van, akkor azt a kép tartósan lefokozott régiójának nevezhetjük. Így azt mondhatjuk, hogy ha a képen sok

nagyon világos pixel van (másnéven fehér), akkor a kép túlexponált és hasonlóképpen, ha nagy mennyiségű sötét pixel van (fekete), akkor a kép alulexponált. [14]

A szakirodalomban gyakran feltételezik, hogy az ideális hisztogram diagramjának hasonlóan kell lennie a normális eloszlás diagramjához. Ez azt jelenti, hogy nem tartalmazhat "sávdigramokat", sem a balszélén (alulexponált), sem a jobb szélén (túlexponált). Az ábrán egy ilyen hisztogram példája látható. A kutatások rámutatnak arra, hogy a hisztogram területe három régióra osztható: világos, sötét és egyéb szürkeárnyalatok. Analóg módon az exponálás is három kategóriába sorolható: alulexponálás (sötét pixelek túlsúlya), túlexponálás (világos pixelek túlsúlya) és a megfelelő exponálás. Természetesen hangsúlyozni kell, hogy a hisztogramdiagramnak nem kell mindig ideális mutatónak lennie a kép minőségének értékelésére. [14]

A hisztogramokat sokféleképpen használják a képfeldolgozásban. Az első felhasználási mód, ahogy azt fentebb is tárgyaltuk, a kép elemzése. Egy képről már a hisztogram megnézésével is jósolhatunk. Ez olyan, mintha egy test csontjának röntgenfelvételét néznénk. Második felhasználási módja a fényerősség. A hisztogramokat széles körben alkalmazzák a kép fényerejében. Nemcsak a fényerősség, hanem a hisztogramokat a kép kontrasztjának beállítására is használják. Ezenkívül másik fontos felhasználási módja a kép kiegyenlítése. Végül, de nem utolsósorban a hisztogramot széles körben használják a küszöbértékek meghatározásánál. Ezt leginkább a számítógépes látásban használják. [14]

2.4 Klasszikus hisztogram kiegyenlítés

A hisztogram kiegyenlítés célja a kép kontrasztjának növelése oly módon, hogy az intenzitásértékek eloszlását egyenletesebbé teszi. Ez a módszer különösen jól alkalmazható olyan képeknél, ahol a dinamikataromány korlátozott, például gyenge fényviszonyok között készített vagy alacsony kontrasztú képek esetében. [16]

A kép egyes szürkeárnyalati értékeihez tartozó gyakoriságokat egy hisztogram írja le. Az egyenlítés célja, hogy ez a hisztogram minél inkább közelítsen az egyenletes eloszláshoz. Ez úgy érhető el, hogy minden intenzitásértékhez hozzárendeljük a halmozott eloszlásfüggvény (CDF) értékét és az új képet e szerint képezzük. Az eredmény egy olyan kép, ahol a sötét és világos árnyalatok egyaránt hangsúlyosabbak, így több részlet válik láthatóvá. [16]

Az alapvető módszer először a kép intenzitás hisztogramjának kiszámítása, amely megmutatja hány pixel tartozik az egyes szürkeárnyalat értékekhez. Ezt követően létre kell hozni egy kumulatív eloszlásfüggvényt, amely megadja, hogy egy adott szint alatti pixelek milyen

arányban találhatóak meg a képen. Végül minden eredeti intenzitásértéket lecserélünk egy új értékre a CDF alapján, ezáltal széthúzva az értékeket a teljes intenzitástartományban (0-255). [16]

Ez a folyamat felerősíti a kontrasztot a képen, különösen azokon a régiókon, ahol az intenzitásértékek korábban összezsúfolódtak, Ugyanakkor a módszer hátránya, hogy a zajos képeken a zaj is felerősödhet, valamint túlhúzhatja a már jól exponált régiókat is. [16]

2.5 Kontrasztkorlátozott adaptív hisztogram kiegyenlítés (CLAHE)

A CLAHE a hisztogramkiegyenlítés továbbfejlesztett, adaptív változata, amely a kontrasztjavítást lokálisan, azaz kis képrészleteken végzi. Ez különösen hasznos abban az esetben, ha a kép különböző részeinek eltérő a fényviszonya, mivel így elkerülhető, hogy a globális kiegyenlítés túl erős vagy túl gyenge legyen. [17]

A CLAHE működése során először a kép kisebb blokkokra oszlik és minden blokkon külön-külön végrehajtásra kerül a hisztogramkiegyenlítés. A kiugró intenzitásokat korlátozza egy úgynevezett clip limit paraméter segítségével, ezzel megakadályozva, hogy egy adott intenzitás túl nagy befolyást kapjon. A kontrasztjavított blokkokat ezután simán összefűzi interpolációval, így nem alakulnak ki mesterséges határok a blokkok között. [17]

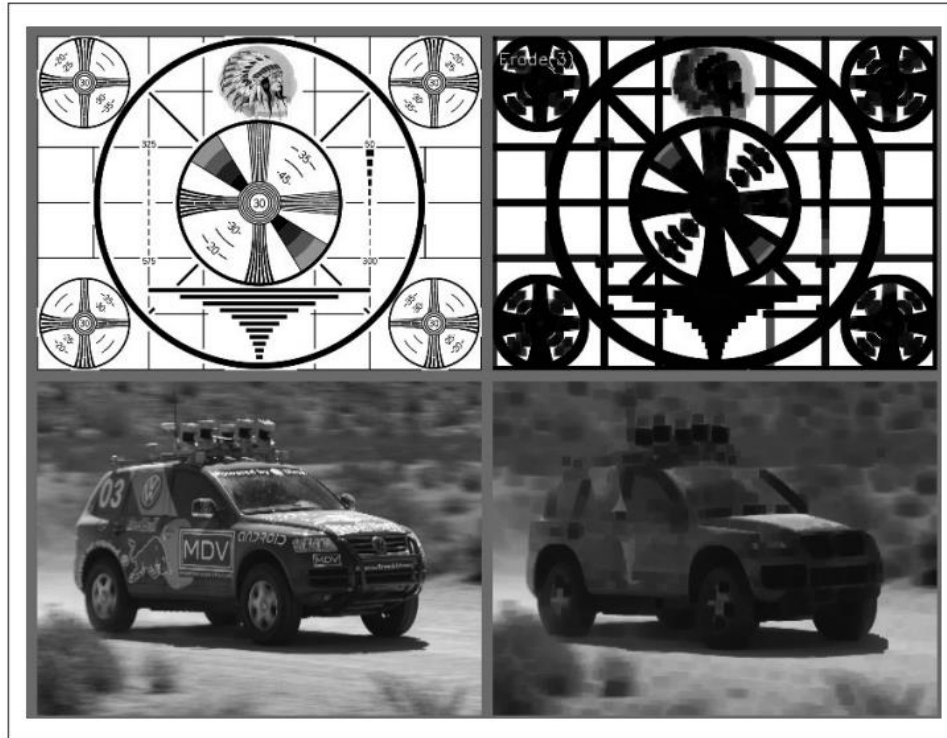
A CLAHE különösen alkalmas orvosi képalkotásban, műholdfelvételeken és minden olyan területen, ahol a lokális kontraszt erősítése szükséges anélkül, hogy a kép túlszűfolt vagy torzított lenne. A módszer érzékenyen reagál a paraméterezésre a vágási határ (clip limit) és a rácsméret (tile grid size) finomhangolásával lehet a kívánt hatást elérni. [17]

2.6 Morfológiai műveletek

A morfológiai transzformációk bináris vagy szürkeárnyaltos képek feldolgozására szolgáló eszközök, amelyek a képen lévő struktúrák (alakzatok, objektumok) formáját elemzik és módosítják. Ezeket a műveleteket gyakran használják zajszűrésre, alakfelismerésre, határvonalak meghatározására és egyéb képfeldolgozási feladatokra. Az alapelv a strukturáló elem más néven kernel vagy maszk használata, amely végig csúszik a képen, és a környező pixelek lokális mintázata alapján új értékeket rendel az adott pixelhez. A műveletek célja lehet az objektumok kicsinyítése, bővítése, leválasztása vagy összekapcsolása a kép kontextusától függően. A két alapvető morfológiai művelet az erózió és dilatació. [16]

Az erózió (7. ábra) célja, hogy úgymond lefaragja az objektumok széleit a bináris képeken. A művelet során egy adott formájú és méretű strukturáló elemet például 3x3-as négyzetet végig

csúsztatunk a képen. Egy pixel akkor marad meg fehérként, aminek értéke 1, ha a strukturáló elem minden pozíciója is fehér a kép adott környezetében. Ellenkező esetben a pixel értékét 0-ra állítjuk. [16]



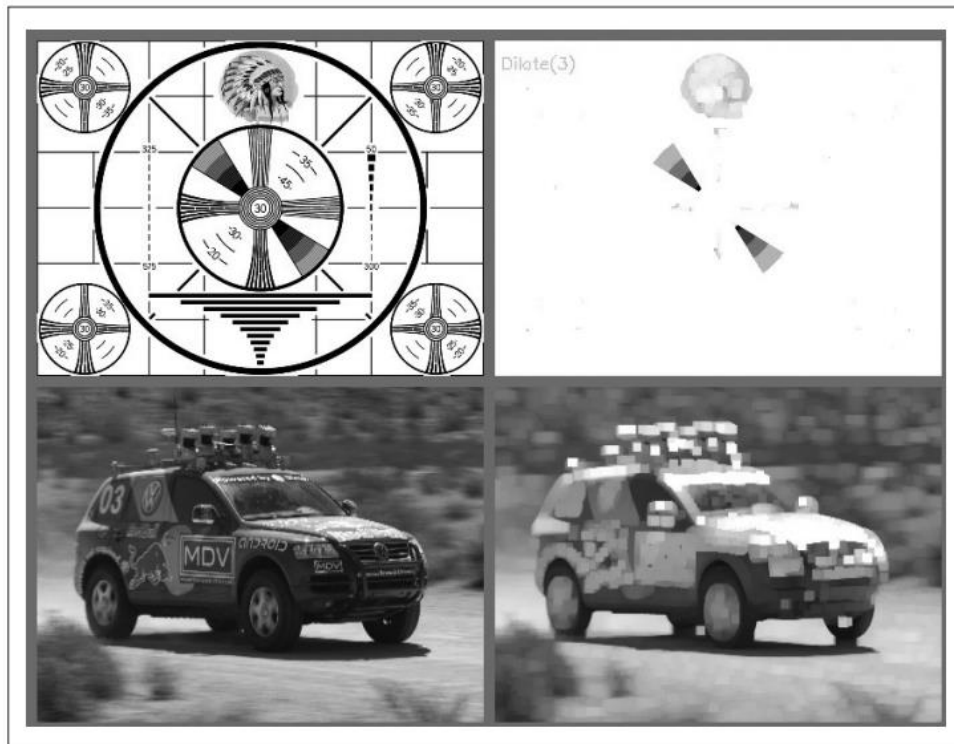
7. ábra: Erózió (balra az eredeti, jobbra az erodált kép: sötét területek növekednek, világos területek csökkennek) [16]

A művelet vékonyítja a világos (fehér) objektumokat, eltávolítja a kisméretű zajokat (elszigetelt fehér pontokat) és szükséges például határok élesítésére vagy objektumok leválasztására. [16]

A dilatació (8. ábra) az erózió ellentéte. A strukturáló elem végig csúsztatása során egy pixel akkor válik fehérre, ha a strukturáló elem legalább egy pozíciója fehér a kép adott környezetében. Ezáltal az objektumok kitágulnak. [16]

Ez a művelet kitölti a fekete pixeleket az objektumon belül, erősíti az objektumok körvonalait és segít az összetartozó részek összekapcsolásában. [16]

Az alapvető morfológiai műveletek, mint az erózió és dilatació, egyszerű, mégis rendkívül hatékony eszközöket biztosítanak az objektumok formájának módosítására bináris és szürkeárnyaltos képeken. Ezek az alpműveletek azonban bizonyos képfeldolgozási feladatok, például a zajeltávolítás, a kis lyukak betömése vagy az objektumok kontúrjának finomítása során önmagukban nem mindig elegendők. [16]



8. ábra: Dilatáció (balra az eredeti, jobbra a dilatált kép: sötét területek csökkennek, világos területek növekednek) [16]

Annak érdekében, hogy bonyolultabb geometriai és topológiai átalakításokat is végre lehessen hajtani, az alapl műveleteket kombinálták. Így jöttek létre az összetett morfológiai műveletek, mint például a nyílás (opening) és a zárás (closing), amelyek az alapvető erózió és dilatáció megfelelő sorrendben történő alkalmazásából származnak. Ezek a kombinált műveletek lehetővé teszik a kép finomabb manipulációját, például a kis méretű zajok eltávolítását vagy a kis szakadások, lyukak kitöltését az objektumokban, anélkül, hogy azok alapvető szerkezete jelentősen megváltozna. [16]

Az összetett morfológiai műveletek tehát az alapl műveletek természetes kiterjesztései, amelyek célja, hogy a képfeldolgozás során még rugalmasabb és hatékonyabb eszközöket biztosítsanak a különféle képi problémák megoldására. [16]

A nyitás egy eróziót követő dilatáció. Ezt a sorrendet alkalmazva az erózió először eltávolítja a kis méretű világos zajokat, majd a dilatáció visszaállítja az objektumok eredeti méretét, de zajok nélkül. [16]

Használható zajsűréshez fehér objektumok vagy két közeli objektum szétválasztása esetén. Például szkennelt dokumentumokon lévő fekete fehér foltok kiszűrésére. Használatával a kis fehér zajok eltűnnek és a lényegi objektumok megmaradnak. [16]

A zárás egy dilataciót követő erózió. Itt először a dilatació kitölti a sötét hiányokat vagy réseket az objektumban, majd az erózió visszaállítja az objektumok eredeti méretét a kitöltött lyukakkal együtt. [16]

Használható kis fekete zajok eltávolítására vagy lyukak betöltésére az objektumon belül. Például szövegben vagy alakzatban lévő apró hiányosságok foltozása. Használatával az objektumok kontúrjai lezáródnak, a belső lyukak megszűnnek. [16]

2.7 Laplacian operátor

A Laplacian-operátor egy derivált operátor, amelyet a kép éleinek megtalálására használnak. A Laplacian és más operátorok, mint például a Prewitt, Sobel, Robinson és Kirsch operátorok között az a fő különbség, hogy ezek mind első rendű derivált maszkok, de a Laplacian egy másodrendű derivált maszk. Ebben a maszkban két további osztályozás van, az egyik a pozitív Laplacian-operátor, a másik pedig a negatív Laplacian-operátor. [15]

Egy másik különbség a Laplacian és a többi operátor között az, hogy a többi operátortól eltérően a Laplacian nem veszi ki az éleket egy adott irányba, hanem a következő osztályozásban veszi ki az éleket. [15]

- Befelé irányuló élek
- Kifelé irányuló élek

A pozitív laplacian operátorban van egy standard maszk, amelyben a maszk középső elemének negatívnak kell lennie, és a maszk sarokelemeinek nullának kell lennie. [15]

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (1)$$

A pozitív Laplacian operátort arra használják, hogy kivegyék a külső éleket a képből. [15]

A negatív Laplacian operátorban is van egy standard maszk, amelyben a középső elemnek pozitívnak kell lennie. A sarokban lévő összes elemnek nullának, a maszk többi elemének pedig -1-nek kell lennie. [15]

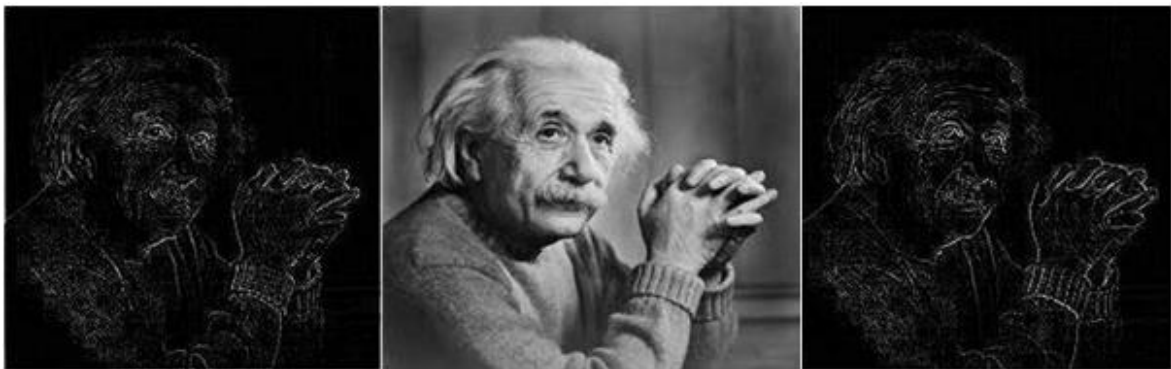
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (2)$$

A negatív Laplacian operátort pedig arra használjuk, hogy kivegyük a befelé irányuló éleket a képből. [15]

A Laplacian operátor használata kiemeli a kép szürkességi szintjeinek megszakadását, és megpróbálja a lassan változó szürkességi szintekkel rendelkező régiókat hangsúlytalanabbá tenni. Ez a művelet olyan képeket eredményez, amelyeken szürkés színű élvonalak és egyéb diszkontinuitások vannak sötét háttéren. Ezáltal a képen befelé és kifelé irányuló élek keletkeznek. [15]

Fontos dolog, hogy hogyan alkalmazzuk ezeket a szűrőket a képre. Ugyanarra a képre nem alkalmazhatjuk a pozitív és a negatív Laplacian operátort. Egy képre csak az egyiket szabad alkalmazni és azt kell megjegyeznünk, hogy ha a pozitív operátort alkalmazzuk a képre, akkor az eredményül kapott képet kivonjuk az eredeti képből, hogy megkapjuk az élesített képet. Hasonlóképpen, ha a negatív operátort alkalmazzuk, akkor az eredményül kapott képet hozzá kell adnunk az eredeti képhez, hogy megkapjuk az élesített képet. [15]

A következő ábrán látható (7. ábra), hogy a szűrők alkalmazása után, hogyan kapunk befelé és kifelé irányuló éleket egy képből:



7. ábra: Példakép (közép) a pozitív operátor alkalmazása után kapott kép (bal) és a negatív operátor alkalmazása után kapott kép (jobb) [16]

A Laplacian-operátor másodrendű derivált maszkot használ, amely nagy érzékenységet mutat a képen található zajokra. Emiatt az operátort gyakran valamilyen előfeldolgozási szűréssel, például Gauss-elmosással együtt alkalmazzák, hogy csökkentsék a zaj hatását a szűrési eredményre. [15]

2.8 Diszkrét Fourier transzformáció

A Diszkrét Fourier-transzformáció (DFT) egy matematikai eszköz, amely lehetővé teszi, hogy egy diszkrét időbeli vagy térbeli jelet (például egy képet) átalakítsunk a frekvenciatartományba. Ez különösen hasznos a képfeldolgozásban, mivel segít azonosítani és manipulálni a kép különböző frekvenciakomponenseit, például az éleket vagy a textúrákat. [16]

A DFT célja, hogy egy adott jelet (például egy képet) felbontson szinusz és koszinusz komponensekre, amelyek különböző frekvenciákat és amplitúdókat képviselnek. Ezáltal a jel frekvenciatartománybeli reprezentációját kapjuk meg, amely gyakran könnyebben értelmezhető vagy feldolgozható bizonyos alkalmazásokban. [16]

Matematikailag a DFT egy N hosszúságú diszkrét jel x_n esetén a következőképpen definiálható: [18]

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N}, k = 0, 1, \dots, N-1 \quad (3)$$

ahol X_k a frekvenciatartománybeli komponensek, x_n az idő- vagy térbeli jel mintái, i pedig az imaginárius egység. [18]

A képfeldolgozásban gyakran használják a következő célokra:

- Élek és textúrák detektálása: A magas frekvenciák az éles változásokat, például éleket jelzik, míg az alacsony frekvenciák a sima területeket. [16]
- Szűrés: A frekvenciatartományban könnyebb bizonyos frekvenciákat kiemelni vagy elnyomni, például zajcsökkentés vagy élesítés céljából. [18]
- Képkompresszió: A segítségével azonosíthatók és eltávolíthatók a kevésbé fontos frekvenciakomponensek, csökkentve ezzel a kép méretét. [18]

A DFT számítási igénye jelentős lehet, különösen nagy méretű képek esetén. Azonban a Gyors Fourier-transzformáció (FFT) algoritmusok lehetővé teszik a DFT hatékonyabb kiszámítását, csökkentve a számítási időt. [16]

Az OpenCV könyvtárban a DFT-t a `cvDFT()` függvénnyel valósíthatjuk meg. Ez a függvény támogatja az egy- és kétdimenziós transzformációkat, valamint lehetőséget biztosít az előre- és visszatranszformációk végrehajtására. A transzformációk során fontos figyelembe venni a bemeneti adatok típusát és méretét, valamint a megfelelő zászlók (flags) beállítását a kívánt művelet eléréséhez. [16]

2.9 Gamma korrekció

A gamma korrekció egy nemlineáris intenzitástranszformáció, amelyet gyakran alkalmaznak digitális képfeldolgozásban a képek megjelenítésének javítása és a szem észlelési karakterisztikájához való illesztése érdekében. Mivel az emberi szem nem egyenletes érzékenységgel reagál a fényerősségre, a gamma transzformáció segít kiegyenlíteni a digitális ábrázolás és az észlelt vizuális élmény közötti eltéréseket. [16][18]

A gamma transzformáció művelete:

$$I = c \cdot I^\gamma \quad (4)$$

ahol I az eredeti intenzitásérték (0 – 255), γ a gamma paraméter, c pedig egy konstans jelölő skálázó tag (gyakran 1). [18]

A transzformáció jellege szerint:

- Ha $\gamma < 1$ A kép világosabb lesz, a sötét részek kiemelése miatt [18]
- Ha $\gamma > 1$ A kép sötétebb lesz, a világos részek csillapítása miatt [18]

A gamma korrekció fontos eszköze a képfeldolgozás során elvégzett előfeldolgozásnak. Számos algoritmus (például élkiemelés, kontrasztjavítás) hatékonyabban működik, ha a kép lineáris fényintenzitás alapján van reprezentálva. Ehhez a gamma korrekció inverzét (linearizálást) is alkalmazzák. [16][18]

Perceptuális szempontból is nagy jelentősége van, mivel az emberi szem jobban megkülönbözteti az alacsony fényintenzitásokat (sötét részek) közötti különbségeket, mint a világos részek esetén. A gamma transzformáció ezt figyelembe véve egyfajta „perceptuális dinamikus tartományt” biztosít, ahol a vizuális információ optimálisan oszlik el a teljes intenzitástartományban. [16][18]

2.10 Point Spread Function

A Point Spread Function (PSF), magyarul Pontterjedési függvény, a digitális képfeldolgozásban azt írja le, hogyan jelenik meg egy ideális pontforrás a képen a rendszer fizikai és optikai korlátai miatt. Ideális esetben egy pontszerű objektumnak egyetlen éles pixelként kellene megjelennie, a valóságban azonban a rendszer tulajdonságai miatt a pont szétként foltként ("blur") jelenik meg. A PSF leírja ezt a szétkenést, azaz a kép és az objektum közötti kapcsolatot, illetve a torzítás mértékét és természetét. [18]

A képdegradáció matematikai modellje alapján a megfigyelt, elmosódott képet az eredeti kép és a PSF konvolúciójaként, zajjal terhelve értelmezzük. Ez a kapcsolat a térbeli tartományban a következőképpen írható fel:

$$g(x, y) = f(x, y) * h(x, y) + n(x, y) \quad (5)$$

ahol $g(x, y)$ a megfigyelt (elmosódott és zajos) kép, $f(x, y)$ az eredeti kép, $h(x, y)$ a PSF, $n(x, y)$ a hozzáadott zaj, $*$ pedig a konvolúció műveletét jelöli. [18]

A frekvenciatartományban ugyanez a kapcsolat a Fourier-transzformáció alkalmazásával szorzásként jelenik meg:

$$G(u, v) = F(u, v) \cdot H(u, v) + N(u, v) \quad (6)$$

ahol $G(u, v)$, $F(u, v)$, $H(u, v)$ és $N(u, v)$ rendre a megfelelő térbeli jelek Fourier-transzformáltjai. [18]

A PSF konkrét formája attól függ, milyen típusú elmosódásról van szó. Mozgásból eredő elmosódás esetén a PSF jellemzően egy vonal menti kiterjedést mutat. Fókuszhibánál a PSF egy kör alakú, enyhén elmosódott korong formájában jelenik meg. Ha pedig atmoszferikus zavarások vagy más sztochasztikus hatások befolyásolják a képet, a PSF ennél komplexebb, diffúz szerkezetet ölthet. Ezeknek a különbségeknek a figyelembevétele létfontosságú a képhelyreállítása algoritmusok, mint például a Wiener dekonvolúció sikeres alkalmazásához.

Gyakorlati helyzetekben a PSF-t ritkán ismerjük pontosan, ezért szükség van annak becslésére. Ha tudjuk az elmosódás típusát, analitikus modellek segítségével is megközelíthetjük a PSF-t. Alternatív megoldás a tapasztalati mérés, ahol ismert minták alapján próbáljuk meghatározni a rendszer viselkedését. A legnehezebb, de legrugalmasabb módszer a vak dekonvolúció, amely során a PSF-t és az eredeti képet egyszerre becsüljük. A helyreállítás minősége szoros összefüggésben áll azzal, hogy mennyire pontos a PSF-re vonatkozó becslésünk. [18]

A Point Spread Function kulcsszerepet játszik a képfeldolgozásban, mivel lehetővé teszi az elmosódás modellezését és kompenzálását. A PSF pontos ismerete vagy becslése alapfeltétele a hatékony képhelyreállítási módszereknek, különösen zajjal terhelt vagy erősen degradált képek esetén. [18]

2.11 Wiener dekonvolúció

A Wiener dekonvolúció egy klasszikus módszer, amelyet elmosódott képek helyreállítására alkalmaznak. A cél az, hogy visszanyerjük az eredeti, éles képet egy olyan elmosódott változatból, amely zajjal is szennyezett lehet. A Wiener szűrő az egyik legismertebb optimális helyreállítási technika, amely statisztikai megközelítést alkalmaz: figyelembe veszi mind a zajt, mind a rendszer választ, amely az elmosódást okozta. [18]

A Wiener dekonvolúció alapját az a feltevés adja, hogy a megfigyelt kép az eredeti kép és egy ismert vagy becsült elmosódási függvény (pontosabban a pontterjedési függvény, PSF) konvolúciójából, valamint additív zajból áll. A módszer a frekvenciatartományban dolgozik, ahol a konvolúció művelet egyszerű szorzássá alakul. A helyreállítás során egy optimalizált szűrőt alkalmazunk, amely minimalizálja a helyreállított kép és az eredeti kép közötti négyzetes hibát, figyelembe véve az elmosódási függvényt és a zaj statisztikai jellemzőit. [18]

A Wiener dekonvolúció matematikai képlete a következő:

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + \frac{S_n(u, v)}{S_f(u, v)}} \cdot G(u, v) \quad (7)$$

ahol $\hat{F}(u, v)$ a helyreállított kép Fourier-transzformációja, $G(u, v)$ az elmosódott kép Fourier transzformációja, $H^*(u, v)$ annak a komplex konjugáltja, $S_n(u, v)$ a zaj teljesítményspektruma, $S_f(u, v)$ pedig az eredeti kép teljesítményspektruma. [18]

Gyakorlati megvalósítás során először a bemeneti képet Fourier-transzformációval átalakítjuk a frekvenciatartományba, majd alkalmazzuk a fenti szűrőt a spektrumokra, végül az eredményt inverz Fourier-transzformációval visszaalakítjuk a térbeli tartományba. A stabilitást biztosító komponens segít elkerülni a túlzott erősítést olyan frekvenciákon, ahol az elmosódási rendszer spektruma közel nullához tart. [18]

A Wiener dekonvolúció egyik legnagyobb előnye, hogy képes jó minőségű helyreállítást biztosítani még zajjal szennyezett képek esetén is, mivel explicit módon modellezi a zajt. A módszer statisztikailag optimális abban az értelemben, hogy minimalizálja az átlagos négyzetes hibát az eredeti és a helyreállított kép között. Emellett, ha a zaj és az elmosódás jellemzői jól ismertek vagy jól becsülhetők, a Wiener szűrő rendkívül hatékonyan működik. [18]

Ugyanakkor a módszernek hátrányai is vannak. Az egyik legnagyobb kihívás a zajspektrum és az elmosódási függvény pontos ismerete vagy becslése, mivel ezek hibás meghatározása jelentősen rontja a helyreállítás minőségét. Továbbá a Wiener dekonvolúció érzékeny lehet a kis frekvenciák környezetében fellépő instabilitásokra, különösen ott, ahol az elmosódási spektrum közel nullához tart. Ilyen esetekben a helyreállított kép zajosnak vagy mesterségesen élesítettnek tűnhet. [18]

Összességében a Wiener dekonvolúció egy elegáns és erőteljes módszer, amely a zajos és elmosódott képek helyreállítására szolgál, különösen akkor, ha a rendszer jellemzői jól ismertek, és a zaj statisztikai viselkedése megfelelően modellezhető. [18]

2.12 Gépi tanulás

A gépi tanulás (machine learning) a mesterséges intelligencia egyik legfontosabb részterülete, amely során a gépek képesek a tapasztalatokból tanulni, és ennek alapján döntéseket hozni, előrejelzéseket adni vagy mintázatokat felismerni. A cél az, hogy a számítógépes rendszerek képesek legyenek az adatokból tanulni, anélkül hogy explicit módon programoznánk minden szabályt. [19][20]

A gépi tanulás interdiszciplináris terület: ötvözi a számítástechnikát, statisztikát, matematikát és sokszor alkalmazás függően a biológiát, pszichológiát vagy mérnöki tudományokat is. A gyakorlatban széles körben alkalmazzák: a keresőmotorok rangsorolásától kezdve a pénzügyi előrejelzéseken és az egészségügyi diagnosztikán át az ipari robotikáig. [19][20]

A tanulás során a rendszer bemeneteket kap, és azokból szabályokat, összefüggéseket próbál

megérteni, majd azokat új helyzetekben is alkalmazni. Ez különösen fontos olyan környezetekben, ahol az adatok mennyisége túl nagy ahhoz, hogy kézi szabályrendszerrel írjuk le. [19][20]

2.12.1 Megközelítései

A gépi tanulási megközelítéseket hagyományosan három nagy kategóriába sorolják, attól függően, hogy milyen jellegű "jel" vagy "visszajelzés" áll a tanuló rendszer rendelkezésére. [20]

Felügyelt tanulás (Supervised learning): A számítógépnek bemeneti példákat és a kívánt kimeneteket mutatnak be, amelyeket egy úgynevezett "tanár" ad meg, és a cél egy olyan általános szabály megtanulása, amely a bemeneteket a kimenetekhez rendeli. [20]

Felügyelet nélküli tanulás (Unsupervised learning): A tanuló algoritmusnak nem adunk címkéket, így ráhagyva azt, hogy megtalálja a struktúrát a bemenetben. A felügyelet nélküli tanulás lehet öncélú (rejtett minták felfedezése az adatokban) vagy eszköz egy cél eléréséhez (jellemzőtanulás). [20]

Erősítéses tanulás (Reinforcement learning): Egy számítógépes program kölcsönhatásba lép egy dinamikus környezettel, amelyben egy bizonyos célt kell teljesítenie (például járművet vezetni vagy egy ellenféllel játszani). Miközben navigál a problémátérben, a program a jutalmakkal analóg visszajelzést kap, amelyet megpróbál maximalizálni. [20]

2.13 Neurális hálózatok

A mesterséges neurális hálózatok (ANN – Artificial Neural Networks) olyan matematikai modellek, amelyek az emberi agy működését utánozó struktúrák alapján tanulnak és végeznek prediktív vagy osztályozási feladatokat. Ezek a hálózatok mesterséges neuronokat tartalmaznak, amelyek kapcsolódnak egymáshoz, és súlyozott jeleket továbbítanak. [21]

A hálózatok alapvetően háromféle rétegből épülnek fel. A bemeneti réteg fogadja az adatokat, például egy kép pixeleit vagy más mért értékeket. A rejtett rétegek – melyekből több is lehet – dolgozzák fel ezeket az adatokat különféle matematikai műveletek és aktivációs függvények (pl. ReLU, sigmoid, tanh) segítségével. A kimeneti réteg adja vissza az eredményt, ami lehet egy osztálycímke (pl. "kutya" vagy "macska"), vagy egy folytonos érték (pl. árbecslés). [21]

A neurális hálózatokat viselkedésük alapján is osztályozhatjuk. Az előrecsatolt hálózatokban az információ kizárólag egy irányba halad: a bemenet felől a kimenet felé, visszacsatolás nélkül. Ezzel szemben a visszacsatolt hálózatok (RNN – Recurrent Neural Networks) képesek korábbi

kimeneteiket is figyelembe venni, így időbeli vagy sorozatfüggő adatokat is hatékonyan feldolgoznak. [21]

2.13.1 Tanítás

A mesterséges neurális hálózatok tanítása során a cél az, hogy a hálózat súlyait úgy állítsuk be, hogy a lehető legpontosabban tudjon válaszolni a bemenetekre. Ehhez általában nagy mennyiségű tanítóadatra van szükség, amelyekhez ismert kimeneti értékek tartoznak (felügyelt tanulás esetén). [22]

A tanítás alapja egy veszteségfüggvény (pl. négyzetes hiba – MSE, vagy keresztentropia – cross-entropy), amely azt méri, mennyire tér el a hálózat kimenete a kívánt céltól. A cél ennek a függvénynek a minimalizálása, amit általában gradiensalapú optimalizálással érnek el. A legelterjedtebb módszer a backpropagation algoritmus, amely a hibát visszaterjeszti a hálózaton, és frissíti a súlyokat az egyes rétegek között. [22]

Fontos lépés a tanítási adathalmaz kezelése. Az adatokat gyakran három részre osztják: [22]

- Tanítóhalmaz (training set): ezzel történik a tényleges tanulás.
- Validációs halmaz (validation set): a modell finomhangolására szolgál.
- Tesztkészlet (test set): a hálózat általánosító képességét méri ismeretlen adatokon.

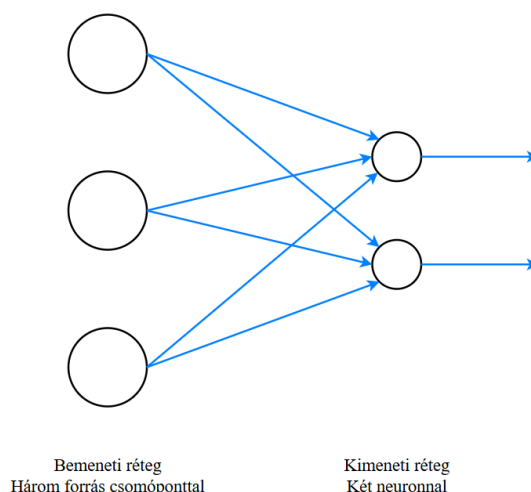
A jó tanítási folyamat eredményeképp a modell nemcsak a tanult mintákat ismeri fel, hanem képes új, hasonló adatokra is helyesen reagálni – ezt nevezzük általánosításnak. [22]

2.13.2 Előrecsatolt neurális hálózatok

Az előrecsatolt hálózatok a neurális hálózatok legegyszerűbb formái, ahol az információ kizárólag egyirányú, előrehaladó kapcsolatban terjed a bemenettől a kimenetig. Ezekben nincs ciklikus kapcsolat, azaz a rendszer nem emlékszik a korábbi bemenetekre. [21]

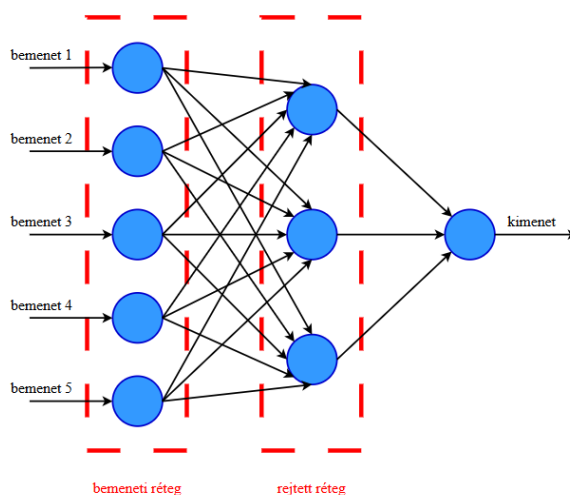
A legegyszerűbb változat az egyrétegű perceptron (9.ábra), amely csak bemeneti és kimeneti rétegből áll. Ez kizárólag lineárisan szeparálható problémák esetén működik megfelelően. Amikor a problémák komplexebbé válnak, szükség van többrétegű perceptronokra (MLP) (10. ábra), amelyek legalább egy vagy több rejtett réteget tartalmaznak. Ezek képesek a bemeneti adatok nemlineáris transzformációjára is, így sokkal összetettebb mintázatok felismerésére alkalmasak. [21]

Ezekben a hálózatokban általában aktivációs függvényeket alkalmaznak a rejtett rétegek után, hogy a modell képes legyen komplex, nemlineáris összefüggések tanulására. Ilyen függvény például a ReLU, a sigmoid vagy a tanh. [21]



9. ábra: Egyrétegű előrecsatolt neurális hálózat – A bemeneti rétegből közvetlen kapcsolat vezet a kimenethez, rejtett réteg nélkül [21]

A tanítás során a hálózat a backpropagation algoritmus segítségével optimalizálja a súlyokat. Ez az algoritmus visszaterjeszti a hibát a hálózat rétegein keresztül, és minden súlyt annak megfelelően módosít, hogy csökkentse a teljes hiba mértékét. [21]



10. ábra: Többrétegű előrecsatolt neurális hálózat – Legalább egy rejtett réteggel bővül az architektúra, lehetővé téve komplex mintázatok tanulását [21]

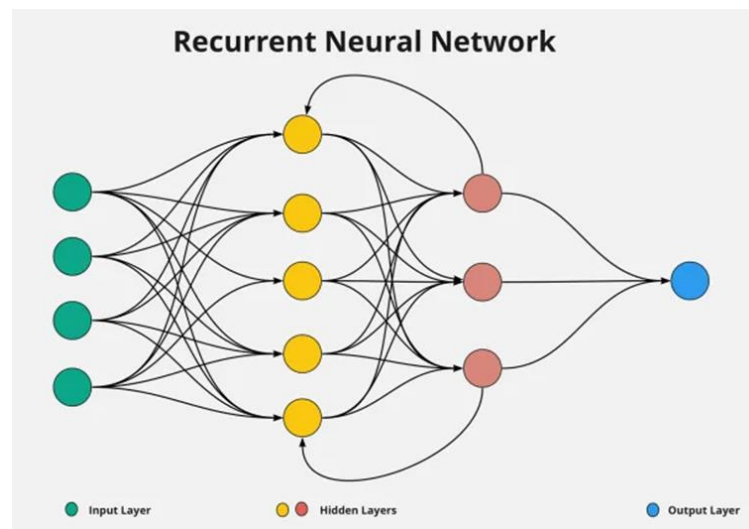
2.13.3 Visszacsatolt neurális hálózatok

A visszacsatolt neurális hálózatok (RNN-ek, 11. ábra) abban különböznek az előrecsatolt hálózatoktól, hogy képesek információt tárolni a korábbi bemenetekből, így figyelembe veszik az időbeli összefüggéseket. Ez különösen fontos sorozatadatok (pl. szövegek, időjárási adatok, beszédfelvételek) esetén. [21]

A leggyakoribb egyszerű változat az RNN, amely minden időlépésben saját korábbi kimenetét

is visszacsatolja bemenetként. Ez azonban gyakran szenved az ún. gradiens eltűnésének problémájától, vagyis nem képes hosszú távú függőségeket megjegyezni. [21]

E probléma kezelésére fejlesztették ki a LSTM (Long Short-Term Memory) és GRU (Gated Recurrent Unit) architektúrákat. Ezek bonyolultabb belső struktúrákkal rendelkeznek, amelyek speciális kapuk segítségével szabályozzák, milyen információ maradjon meg, és mi törlődjön. Ezáltal lehetővé teszik, hogy a hálózat hosszabb távú összefüggéseket is megtanuljon, például mondatértelmezés során vagy prediktív idősortanulási feladatokban. [21]



11. ábra: Egyszerű visszacsatolt neurális hálózat – A rendszer saját előző állapotát is figyelembe veszi a feldolgozás során [39]

2.14 Konvolúciós neurális hálózatok

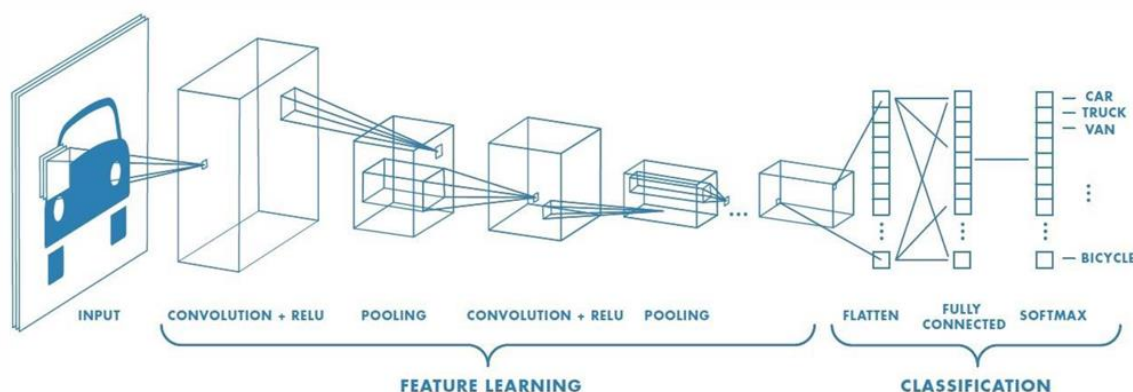
A konvolúciós neurális hálózatok (CNN-ek, 12. ábra) különösen hatékonyak képfeldolgozási feladatoknál, mivel képesek automatikusan felismerni a képekben rejlő mintázatokat. Ahelyett, hogy előre meghatározott jellemzőket használnának, a CNN-ek saját maguk tanulják meg, hogy mely jellemzők fontosak a feladat szempontjából. [23]

A hálózat első rétegei általában **konvolúciós rétegek**, amelyek kis méretű szűrőket (pl. 3×3 vagy 5×5) alkalmaznak a képre. Ezek a szűrők képesek kiemelni különböző jellemzőket, például éleket, sarkokat vagy textúrákat. A következő lépés a **pooling réteg**, amely a jellemzőtér méretét csökkenti, miközben megőrzi a fontos információkat – ezzel csökken a számítási igény és a túlillesztés esélye is. [23]

A több konvolúciós és pooling réteg után következik a **flatten réteg**, amely a többdimenziós jellemzőket egy hosszú vektorra alakítja, amit végül egy vagy több **fully connected réteg** dolgoz fel. Ez a rész a döntéshozatalért felelős, és jellemzően ugyanúgy működik, mint a

klasszikus többrétegű perceptronok. [23]

A CNN-ek nagy előnye, hogy kevesebb paraméterrel dolgoznak, mivel a szűrők megosztott súlyokat használnak. Ez hatékonyabb tanulást és jobb általánosítást tesz lehetővé. Alkalmazásaik széles körűek: képosztályozás (pl. ImageNet), objektumdetekció, orvosi képfeldolgozás, önvezető autók képi elemzése és sok más terület. [23]



12. ábra: Konvolúciós neurális hálózat felépítése [40]

2.15 Hasonló munkák

Ebben a fejezetben olyan már meglévő módszereket szeretnék bemutatni, amelyek segítségével megvalósítható az általam kiválasztott projekt.

2.15.1 Túlexponálás korrigálása és detektálása

A képek expozíciós hibáinak – alul- vagy túlexponálásának – detektálására a kutatások jellemzően a hisztogramok elemzését használják. Az alulexponált képeken a hisztogram elsősorban a sötét tartományokban koncentrálódik, míg túlexponált képek esetén a világos tartományok dominálnak. Egyes megközelítések küszöbértékek alapján számítják a túl sötét és túl világos pixelek arányát. Egy konkrét módszer a Brightness Preserving Bi-Histogram Equalization (BBHE), amelyet Kim alkalmazott 1997-ben [24]: ez a megközelítés a képet két részre osztja a középérték alapján, és mindkét részre külön hisztogramkiegyenlítést alkalmaz a fényesség megőrzése érdekében. A javításra emellett gyakran alkalmaznak CLAHE-t (Contrast Limited Adaptive Histogram Equalization), amelyet Zuiderveld vezetett be 1994-ben [25], és az egyenletesebb kontraszteloszlás elérésére törekszik, miközben megakadályozza a túlzott kontraszterősítést.

2.15.2 Elmosódások korrigálása és detektálása

A homályosság detektálására több klasszikus módszer is létezik, például a Laplace-variancia

számítása, ahol az alacsony variancia a részletek hiányára, azaz homályosságra utal. Egy konkrét megoldás a vak dekonvolúció (Blind Deconvolution), amelyet Levin és munkatársai alkalmaztak 2007-ben [26]: a PSF (Point Spread Function) becslésével próbálja meg visszaállítani az eredeti éles képet anélkül, hogy előzetesen ismernénk az elmosódás típusát. Javításra emellett gyakran alkalmazzák a Wiener-dekonvolúciót is, amely különösen hasznos, ha zaj is jelen van a képen.

2.15.3 Piros szem detektálása és korrigálása

A piros szem effektus detektálása jellemzően arc- és szemdetektor algoritmusokon alapul (például Haar Cascade Classifiers). Egy konkrét módszer például Luo és munkatársai 2003-as megközelítése [27], amelyben a szín- és formainformáció együttes alkalmazásával először detektálják a szemeket, majd a piros színintenzitás alapján szegmentálják a hibás területeket. A korrekció során a vörös régió színeit természetesebb árnyalatokkal helyettesítik úgy, hogy a kék és zöld csatornák átlagából számított értékkel pótolják a vörös csatornát, megőrizve a szem eredeti textúráját és részleteit.

2.15.4 Purple Fringe detektálása és korrigálása

A purple fringing hibát gyakran magas kontrasztú élek körül jelenik meg, ahol a kromatikus aberráció jelentős. A detektálás tipikusan éldetektáláson (például Sobel operátorral) és színinformáción alapul. Egy konkrét megoldás az, amelyet Gharbi és munkatársai javasoltak 2015-ben [28]: azokon a területeken, ahol a kék és piros csatorna intenzitása meghalad egy küszöbértéket, lila szín jelenlétére lehet következtetni. A morfológiai műveletekkel finomított maszk alkalmazása után fokozatosan csökkentik a szín telítettségét és fényerejét gamma korrekció segítségével, majd újraszámítják a helyes színegyensúlyt a szomszédos pixelek átlagértékei alapján.

2.15.5 Képosztályozás

A képosztályozás célja a képek automatikus kategorizálása adott osztályokba. Hagyományosan a képi jellemzők (például színeloszlás, textúra) extrakciójával és klasszikus gépi tanulási algoritmusokkal (SVM, kNN) történt a besorolás. Egy konkrét megoldás például a LeCun és munkatársai által 1998-ban bemutatott CNN-alapú (Convolutional Neural Network) képosztályozás [29], ahol a nyers képadatokból a hálózat automatikusan tanulja meg a megfelelő reprezentációkat, és rétegenként egyre absztraktabb jellemzőket használ fel az osztálycímkék pontos meghatározásához.

2.16 Fejlesztő környezet, programnyelv, programcsomagok

A program a Python nyelvre épül, amely napjaink egyik legnépszerűbb, nyílt forráskódú, magas szintű programozási nyelve. A Python könnyen olvasható szintaxisa és hatékony könyvtártámogatása ideálissá teszi tudományos, képfeldolgozási és gépi tanulási alkalmazásokhoz egyaránt. A Python nyelv népszerűségét nagyban erősíti a közösségi támogatás és az, hogy számos területen – köztük a digitális képfeldolgozásban is – kulcsfontosságú csomagokat kínál. [30]

A program fejlesztése és futtatása egy grafikus operációs rendszerrel rendelkező gépen történt, amelyen a Python 3-as verzióját használtuk. A fejlesztőkörnyezet szerepét Visual Studio Code töltötte be, amely rugalmas konfigurálhatóságával és kiegészítőivel megkönnyíti a Python-fejlesztést, különösen a nagyobb, moduláris alkalmazások esetén. [31]

A képfeldolgozási feladatok ellátásához elsősorban az OpenCV könyvtárra támaszkodtam, amely széleskörű funkciókat biztosít képek beolvasására, megjelenítésére, manipulálására, valamint különféle transzformációkra és szűrésekre. Az OpenCV C++ és Python interfésszel is elérhető, így a Pythonban való felhasználása különösen kényelmes a gyors prototípus-fejlesztés során. [16]

A program működéséhez további fontos csomagokat is használtam:

- NumPy: hatékony numerikus tömbkezelést biztosít, amely a képadatok mátrixszintű manipulációjához nélkülözhetetlen. Különösen fontos szerepet játszik a Fourier-transzformáció és egyéb alacsony szintű matematikai műveletek során. [32]
- Matplotlib: a képek és hisztogramok vizualizálásához szükséges grafikus könyvtár, amely lehetővé teszi a képadatok vizuális ellenőrzését és kiértékelését. [33]
- Pillow (PIL): képek betöltésére, átméretezésére, mentésére használt csomag, amely egyszerű felületet kínál az alapszintű képműveletekhez. [34]
- imagehash: a képduplikátumok felismerésében játszik szerepet. A könyvtár lehetővé teszi a vizuálisan hasonló képek összehasonlítását különféle perceptuális hash algoritmusok segítségével. [35]
- Tkinter és CustomTkinter: ezek biztosítják a program grafikus felhasználói felületét (GUI), amelyen keresztül a felhasználó mappát választhat, képeket tekinthet meg, valamint elindíthatja a detektálási és javítási lépéseket. A Python beépített Tkinter könyvtára egyszerű, gyors fejlesztést tesz lehetővé, míg a CustomTkinter a felület modernizálását segíti elő. [36]

- TensorFlow / Keras és PyTorch: mélytanulási modellek betöltésére és használatára szolgálnak. Ezekre a homályosság típusának osztályozásához és a PSF-alapú javításhoz volt szükség, amelyhez egyedi tanított modelleket alkalmaztam. [37][38]
- Ultralytics YOLOv8: az osztályozási lépéshez, valamint az arcdetektáláshoz és a piros szem hiba előfeldolgozásához az Ultralytics által fejlesztett, könnyen integrálható és valós idejű objektumdetektálásra optimalizált YOLOv8 modellt használtam. A modell hatékonyan képes arcokat és szemeket detektálni különböző fényviszonyok között, és testreszabható különböző osztályozási feladatokra is. [46]
- pandas: az osztályozási modul során a képekhez tartozó jellemzők, maszkok és osztályozási eredmények hatékony táblázatos kezelésére szolgál. A pandas lehetővé teszi az adatstruktúrák gyors szűrését, rendezését és kiértékelését, így ideális eszköz a képekhez tartozó metaadatok vagy statisztikák kezelésére. [47]

3 Megvalósítás

A projekt célja egy olyan szoftver elkészítése volt, amely képes különböző képalkotó eszközök segítségével készített képeken található képalkotási hibák detektálására, javítására és osztályozására. Az osztályozás a képeken szereplő látványvilág alapján történik.

A szoftver működése a következő folyamatokra osztható fel:

1. Képeket tartalmazó mappa kiválasztása és előfeldolgozása
2. A beolvasott képeken található hibák detektálása
3. Hibák javítása és a javított képek mentése
4. Képek osztályozása és fotóalbumba rendszerezése mesterséges intelligencia segítségével

Az első folyamat egy része egy külön programfájlban található (`folderselect.py`) ez tartalmazza a képeket tartalmazó mappa kiválasztását és a hozzá tartozó grafikus felületet. Az előfeldolgozás és a további folyamatok egy másik programfájlban (`photoselector.py`) történnek, valamint ezt segíti néhány segédprogram, amelyek függvényeket tartalmaznak a hibák detektálása folyamathoz (`faults_detection.py`) vagy például a konvolúciós neurális hálózat betanítását, amely az osztályozáshoz szükséges (`yolov8_train.py`).

3.1 Program használatának előfeltételei

Az elkészült szoftver megfelelő működésének van néhány előfeltétele, aminek meg kell felelnie a képeknek és a képeket tartalmazó mappának.

A program megfelelő működéséhez szükséges, hogy a képek fájlformátuma `.jpg`, `.png`, vagy `.jpeg`, mivel más fájlformátumú képeket a program a mappából nem fog beolvasni.

A program jelenleg a nagyobb felbontású képeket észrevehetően lassabban kezeli a detektálás és javítás folyamatoknál, ezért nagyobb felbontású képeknél például 4K-s képeknél észrevehető a hosszabb detektálás vagy javítás.

3.2 Segédosztályok konstansok

A program különféle fixen definiált paramétereket (konstansokat) és segédosztályokat használ az egyes képfeldolgozási lépések során. Ezek célja, hogy elősegítsék az adatok tárolását, a felhasználói interakció kezelését, vagy a feldolgozási logikák testreszabását. Emellett több helyen alkalmaz a rendszer dinamikusan kiszámított értékeket is, amelyek a képek jellemzői alapján automatikusan változnak.

A konstansok gyakran az előképzett modellekhez, küszöbértékekhez, osztálynevekhez vagy feldolgozási paraméterekhez kapcsolódnak, míg az osztályok inkább az aktuális képekkel kapcsolatos állapotokat és eredményeket tárolják átmenetileg.

3.2.1 Fixen definiált konstans paraméterek

Ezek a konstansok (1. kódrészlet) a képhibák felismeréséhez (pl. elmosódottság) és javításához (pl. Wiener-dekonvolúció) szükséges modellek beállításait, valamint a küszöbértékeket tartalmazzák.

1. kódrészlet: Fixen definiált konstans paraméterek

```
LAPLACE_THRESHOLD = 1000.0
NOISE_THRESHOLD = 150.0
CLASS_NAMES = ['defocus', 'motion']
RESIZE_HEIGHT = 512
BATCH_SIZE = 8
KERNEL_SIZE = 15
MOTION_MODEL_PATH = "models/psf_predictor_motion.pth"
DEFOCUS_MODEL_PATH = "models/psf_predictor_defocus.pth"
```

3.2.2 Dinamikusan számolt paraméterek

A feldolgozás során több olyan érték is szerepel, amely a bemeneti képek tartalmától függően kerül meghatározásra. Ezek az értékek lehetővé teszik a képadaptív zajcsökkentést, kontrasztjavítást és gamma korrekciót, így a program automatikusan alkalmazkodik az aktuális képhez.

2. kódrészlet: Dinamikusan számolt paraméterek

```
if brightness < 70:
    clip_limit = 4.0
elif brightness < 120:
    clip_limit = 3.0
elif brightness > 200:
    clip_limit = 1.5
elif brightness > 160:
    clip_limit = 2.0

if brightness < 80:
    gamma_value = 1.5
    apply_gamma = True
elif brightness > 180:
    apply_gamma = False
else:
    gamma_value = 1.2
    apply_gamma = True
```

3.2.3 Segédosztályok

Ezek az osztályok strukturált módon tárolják a képfeldolgozás során keletkező állapotokat és eredményeket. Kiemelten fontos szerepet játszanak a GUI és a háttérben zajló feldolgozás közötti kommunikációban.

3. kódrészlet: Segédosztályok

```
class canvas_data:
    canvas_sizex = 830
    canvas_sizey = 560

class image_typ:
    purple_img = []
    hist_img = []
    repaired_img = []
    original_img = []

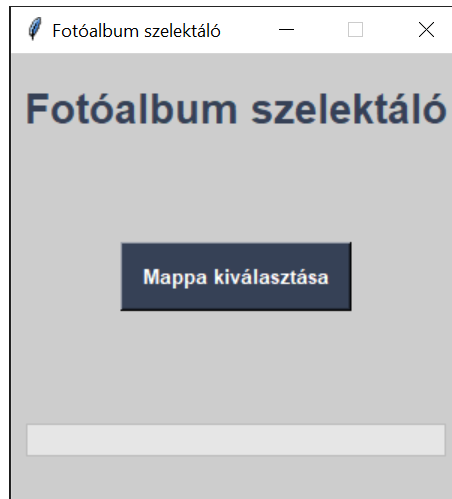
class checkbox:
    under_over_exposure = tk.IntVar()
    blur = tk.IntVar()
    red_eye = tk.IntVar()
    purple_fringe = tk.IntVar()
```

3.3 Grafikus felület

A felhasználók számára a grafikus felületek sokkal érthetőbbek, látványosabbak, mint a szöveg alapú felületek, ahol a megjelenített vizuális elemek a felhasználó számára releváns információkat, valamint az általa elvégezhető műveleteket közvetítik. A grafikus felület segítségével gyorsabban és könnyebben végezhetőek el feladatok kódolási nyelv és számítógépes parancsok ismerete nélkül is. Ezen jellemzők ismeretében készítettem a szoftverhez grafikus felületet.

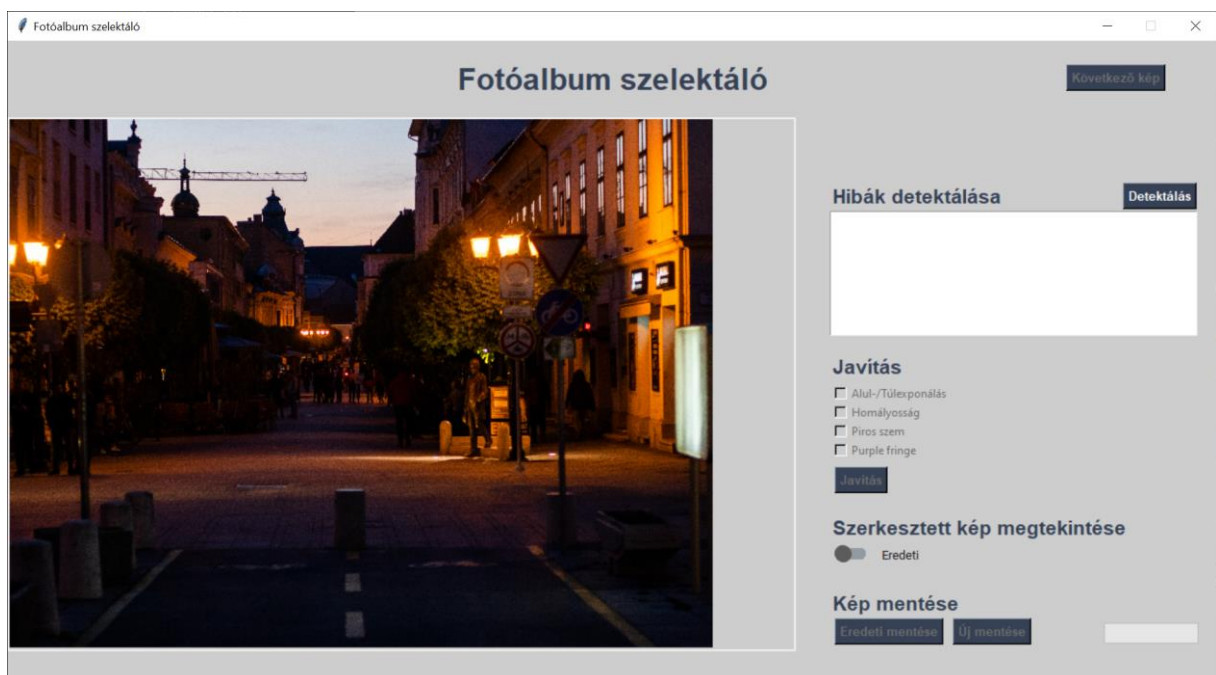
A program grafikus felülete két ablakból áll. A program indításakor egy kisebb méretű ablak jelenik meg, amely a képeket tartalmazó mappa kiválasztását segíti a felhasználó számára, hogy az elvégezhető legyen egy konkrét elérési útvonal megadása helyett.

Az első felületen (13. ábra) egy „Mappa kiválasztása” névvel rendelkező gomb található, amellyel a felhasználó kiválaszthatja azt a mappát, ami a szerkesztésre váró képeket tartalmazza. A mappa kiválasztása után egy úgynevezett folyamatjelző sáv jelzi a felhasználó számára, hogy hogyan halad a kiválasztott mappában található fájlok előfeldolgozása.



13. ábra: Mappa kiválasztási felület.

A második felület (14. ábra) a képeket tartalmazó mappa kiválasztása után automatikusan betöltődik. Ezen felületen már többféle funkcionális elem például gomb, jelölőnégyzet, szövegdoboz és képmegjelenítő található. Itt tudja a felhasználó a gombok segítségével elvégezni a képalkotási hibák detektálását, javítását, mentését és osztályozását. A felhasználói élmény növelése érdekében a felhasználó folyamatosan visszacsatolást kap a program működéséről egy képmegjelenítőn és egy szöveges felületen keresztül. A felületen helyet kapott még egy úgynevezett érintősáv is, amely segítségével a felhasználó össze tudja hasonlítani az eredeti és szerkesztett képet mentés előtt.



14. ábra: Főfelület

3.4 Képek betöltése és előfeldolgozása

Ez a funkció a felhasználó által kiválasztott mappából tölt be képeket, majd előfeldolgozási lépéseket hajt végre rajtuk. A betöltés után következik a képduplikátumok felismerése, majd ezt követi a duplikátumokon történő lehetséges hibák detektálása, amelyek alapján egy pontozási rendszer segítségével a legjobb minőségű képek kiválasztása történik az egyes duplikátum csoportok közül.

3.5 Mappaválasztás és képek betöltése:

A felhasználó egy dialógusablak segítségével kijelöli a feldolgozandó mappát. A kiválasztott könyvtár elérési útja mentésre kerül, majd a program végig ellenőrzi annak tartalmát. Csak azokat a fájlokat veszi figyelembe, amelyek kiterjesztése .jpg, .png vagy jpeg. Amennyiben a mappa nem tartalmaz ilyen képeket, hibaüzenetet jelenít meg, és új mappa kiválasztását kéri. Ha a mappa tartalmaz megfelelő kiterjesztésű fájlokat, akkor következik a képek előfeldolgozása.

3.6 Előfeldolgozási lépések

Miután a képek beolvasásra kerültek, minden képre zajdetektálás történik, amely a Fourier-spektrumból számolt átlagos magnitúdó alapján történik. Ez a mérőszám alapján választja ki a rendszer a megfelelő zajcsökkentési módszert: erős impulzus zaj esetén medián szűrés, textúrazaj esetén bilaterális szűrés, homogén zajra Gaussian szűrés, míg alacsony zajszintnél nemlokális átlagoló módszer (NLM) alkalmazandó. A zajcsökkentés minden képre külön történik, függetlenül attól, hogy később duplikátumként azonosításra kerül-e.

Ezt követően a képeken hash-alapú összehasonlítás történik a `imagehash.whash()` függvény segítségével. Ez a vizuális hash a képek strukturális jellemzői alapján azonosítja a hasonlóságot. Ha két kép hash-értéke közti különbség kisebb, mint egy előre meghatározott küszöbérték (jelen esetben 5), a képek duplikátumként kezelhetők. Az azonos duplikátumcsoportba sorolt képekre lefut a teljes hibadetektálási és pontozási folyamat.

A legjobb kép kiválasztása pontozási rendszer alapján történik, ahol minden hibamentes tulajdonság (helyes expozíció, nem elmosódott részletek, fringing-mentesség, piros szem hiánya) 100 pontot ér. Ha a pontszám megegyezik, a rendszer további mérőszámokat használ: az élek mennyisége (`edge_score`), a túl világos vagy túl sötét pixelek aránya, valamint a lila elszíneződések (`purple fringing`) mértéke. Ez a döntési folyamat biztosítja, hogy minden duplikátumhalmazból a legjobb minőségű változat kerüljön kiválasztásra további feldolgozás céljából.

A kiválasztott képek bekerülnek az `ImageStore.images` listába, a hozzájuk tartozó fájlnevek pedig az `ImageStore.image_names` listába. Ezek a képek további detektálási, javítási és osztályozási folyamatok alapjai lesznek.

3.7 Képközpontozási hibák detektálása

A program a képeken gyakran előforduló képközpontozási hibákat próbálja meg felismerni. A detektálás eredményéről a felhasználó értesítést kap a grafikus felületen található szövegdobozon keresztül. A képközpontozási hibák detektálása függvények segítségével történik, amelyek a `fault_detection.py` fájlban találhatóak meg.

3.7.1 Expozíció

A képek expozíciós állapota az `estimate_exposure` függvény (4. kódrészlet) segítségével történik. A képek feldolgozása során gyakran előfordul, hogy a felvétel túl sötét (alul exponált) vagy túl világos (túlexponált), ami rontja a vizuális minőséget és az utófeldolgozás hatékonyságát. Ez a függvény a kép fényességi eloszlását elemzi, különböző szempontok alapján, hogy objektív módon meghatározza, melyik kategóriába sorolható a kép.

A függvény bemeneti paramétere egy színes kép numpy tömb formátumban (BGR szintérrel rendelkezik, amely az OpenCV által támogatott formátum).

A függvény három fő tényezőt vizsgál a kép expozíciójának meghatározásához:

1. Hisztogram elemzés és a domináns fényességérték meghatározása – A szürkeárnyaltos kép fényességi eloszlásából a leggyakoribb fényességi értéket (módusz) keresi meg.
2. Átlagos fényesség kiszámítása – A teljes kép fényességeloszlását figyelembe véve meghatározza az átlagos fényességi értéket.
3. Sötét és világos pixelek arányának vizsgálata – Az alacsony (<50) és magas (>205) fényességi értékek arányát méri a teljes képen belül.

Ezek alapján a függvény osztályozza a képet az alábbi kategóriák egyikébe:

- Alul exponált (1): Ha a kép leggyakoribb fényességi értéke alacsony (≤ 120), az átlagos fényessége szintén alacsony (< 100), és a sötét pixelek aránya nagyobb mint 40%, akkor a kép alul exponáltnak tekinthető.
- Megfelelő expozíció (0): Ha a leggyakoribb fényességi érték 120 és 160 között van, az átlagos fényesség 100 és 180 között mozog, és nincsen kiugróan nagy arányban sötét vagy világos pixel, akkor a kép jól exponált.

- Túlexponált (2): Ha a leggyakoribb fényességi érték magas (≥ 160), az átlagos fényesség is nagy (> 180), és a világos pixelek aránya meghaladja a 40%-ot, akkor a kép túlexponáltnak tekinthető.

A függvény eredményeként egy logikai értéket (True vagy False) és egy egész számot ad vissza, amely az expozíciós osztályozást jelöli.

4. kódrészlet: Alul- és túlexponálás detektálása

```
def estimate_exposure(image: np.array):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    hist_gray = cv2.calcHist([gray], [0], None, [256], [0, 256])

    max_pixel_brightness_place = int(np.argmax(hist_gray))
    avg_brightness = np.mean(gray)
    dark_pixels = np.sum(gray < 50) / gray.size
    bright_pixels = np.sum(gray > 205) / gray.size

    if max_pixel_brightness_place <= 120 and avg_brightness < 100
    and dark_pixels > 0.3:
        return True, 1, max_pixel_brightness_place,
        avg_brightness, dark_pixels
    elif max_pixel_brightness_place > 120 and
    max_pixel_brightness_place < 160 and 100 <= avg_brightness <=
    180:
        return False, 0, max_pixel_brightness_place,
        avg_brightness, bright_pixels
    elif max_pixel_brightness_place >= 160 and avg_brightness >
    180 and bright_pixels > 0.3:
        return True, 2, max_pixel_brightness_place,
        avg_brightness, bright_pixels
    else:
        return False, 0, max_pixel_brightness_place,
        avg_brightness, bright_pixels
```

3.7.2 Elmosódás

A képek elmosódottságának megállapítása `estimate_blur` függvény (5. kódrészlet) segítségével történik. A függvény feladata annak eldöntése, hogy a bemeneti kép homályos vagy éles. Amennyiben a kép homályos, a függvény annak típusát is meghatározza egy mélytanuláson alapuló osztályozó modell segítségével.

Ez a kétlépcsős rendszer ötvözi a hagyományos képfeldolgozási technikákat a gépi tanulással, és pontosan képes felismerni az életlenség meglétét és típusát.

1. Laplace-variancia alapú homályosságvizsgálat

Az első lépésben a rendszer a kép élességét a Laplace-operátor varianciájával méri.

A számítás előtt a kép szürkeárnyaltosra konvertálása után újra méretezés történik a megadott konstans érték `RESIZE_HEIGHT` pixel magasságra, miközben az oldalarányt megtartja. Ez a lépés azért fontos, mert:

- Különböző felbontású képeken a Laplace-variancia nem összehasonlítható.
- A méretezés egységesíti a feldolgozási feltételeket és javítja a küszöbérték alapú döntés pontosságát.

A Laplace-variancia értelmezése a következők szerint alakul:

- Ha a variancia magas, akkor éles a kép
- Ha a variancia alacsony, azaz kisebb a megadott küszöbértéknél, akkor a kép potenciálisan homályos, és a második lépés következik.

5. kódrészlet: Homályosság meghatározása

```
def resize_keep_aspect(image, target_height):
    h, w = image.shape[:2]
    scale = target_height / h
    new_width = int(w * scale)
    return cv2.resize(image, (new_width, target_height))

def laplacian_variance(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    resized = resize_keep_aspect(gray, target_height =
    RESIZE_HEIGHT)
    lap_var = cv2.Laplacian(resized, cv2.CV_64F).var()
    return lap_var

def estimate_blur(image: np.array):
    lap_var = laplacian_variance(image)

    if lap_var < LAPLACE_THRESHOLD:
        detected_type = classify_blur_types(image)
        return True, lap_var, detected_type
```

2. Homály típusának meghatározása (6. kódrészlet) neurális hálóval

Amennyiben a Laplace-variancia alapján a kép homályosnak minősül, egy előre betanított Keras-alapú konvolúciós neurális hálózat segítségével történik a homály típusának meghatározása. A hálózat a következő osztályokba sorolja a képet:

- Defókuszált – amikor a kép fókuszon kívül van
- Mozgási – amikor a kép mozgás hatására lett elmosódott

A neurális háló előfeldolgozási lépésként 512x512 pixeles képeket vár, normálva a [0, 1] tartományba

A homályosság osztályozó modell tanítása a Kaggle Blur Dataset adatkészlettel lett tanítva, amely két mappát tartalmaz: motion és defocus.[41]

6. kódrészlet: Homály típusának meghatározása

```
def classify_blur_types(img):  
    resized = np.array([cv2.resize((img * 255).astype(np.uint8),  
    (512, 512)) / 255.0])  
    preds = blur_classifier.predict(resized),  
    batch_size=BATCH_SIZE, verbose=0)  
    return CLASS_NAMES[np.argmax(preds[0])]
```

3.7.3 Piros szem

Az piros szem hibával rendelkező képek detektálása az `estimate_redeye` függvény (7. kódrészlet) segítségével történik. A függvény feladata a vörös szem effektus automatikus detektálása a bementi képen. A vörös szem hiba leggyakrabban vaku használata során jelentkezik, amikor a fény visszatükröződik a retináról. A függvény a következő lépések segítségével azonosítja az ilyen területeket:

1. Arcok és szemek detektálása a `face_detection` függvény és a Haar-kaszád osztályozó segítségével.
2. Piros szem területek azonosítása a HSV színtérben adott színküszöbök alkalmazásával.
3. Maszk generálása és vizuális ellenőrzés a piros szem területekről.
4. Pixelalapú elemzés és küszöbérték számítás a végső döntés meghozatalához a detektálás eredményéről.

A program először egy alfüggvény segítségével arcfelismerést végez a bemeneti képen, amely a YOLOv8 modell [45] segítségével azonosítja a képen lévő arcokat. Ha nem található arc a piros szem elemzést nem hajtja végre, ami pontosabbá teszi a detektálást.

Ha arcokat talál, a `cv2.CascadeClassifier("haarcascade_eye.xml")` modellt [44] használja a szemek felismerésére. Az `eyeRects` tömb minden szem téglalap koordinátáit és magasságát és szélességét tartalmazza, amely minden eleme `[x, y, w, h]` alakú. Ha nem található szem, a függvény `False`-t ad vissza.

A detektált szemterületeket kivágja az eredeti képből, majd HSV színtérbe konvertálja. Egy előre meghatározott színtartományt használ, hogy létrehozza a piros szem maszkot.

A függvény a maszk alkalmazása után megszámolja a piros pixeleket. Ha ezek száma meghalad egy küszöbértéket (a szem összes pixelének 1%-át), akkor piros szem hibát jelez.

Ha legalább egy szem tartalmaz vörös szem hibát, a függvény igaz értékkel tér vissza. Ellenkező esetben hamis eredményt ad vissza.

7.kódrészlet: Piros szem hiba detektálása

```
def estimate_redeye(image: np.array):
    eyesCascade = cv2.CascadeClassifier("haarcascade_eye.xml")

    eyeRects = eyesCascade.detectMultiScale(image , 1.1, 5)
    eyeRects = eyeRects.astype(np.uint8)
    if len(eyeRects) == 0:
        return False, eyeRects

    detected_redeye = 0
    lowered2 = np.array([160,100,20])
    uppered2 = np.array([179,255,255])

    for x,y,w,h in eyeRects:
        eyeImage = image [y:y+h , x:x+w]
        if eyeImage.size == 0:
            continue
        eyeImage_hsv = cv2.cvtColor(eyeImage, cv2.COLOR_BGR2HSV)

        mask = cv2.inRange(eyeImage_hsv, lowered2, uppered2)

        cropped = cv2.bitwise_and(eyeImage, eyeImage, mask=mask)

        redpixel_counter = np.count_nonzero(cropped)
        allpixel_count = eyeImage.size
        threshold = allpixel_count * 0.01
        if redpixel_counter > threshold:
            detected_redeye += 1

    return detected_redeye > 0, eyeRects
```

3.7.4 Purple Fringe

A képeken előforduló lila rojtosodás hibák detektálása `purple_fringe_detection` függvény segítségével történik. A függvény szín- és élapú technikák kombinációjával végzi a detektálást és opcionálisan visszaadja a hibás területeket tartalmazó bináris maszkot. A hiba detektálása a következő lépésekben történik:

1. Színcsatornák szétválasztása és színkülönbség kiszámítása

A függvény első lépésként szétválasztja a BGR színcsatornákat. Ezután kiszámítja a vörös és kék csatornák különbségének abszolút értékét (`diff_rb`). A nagy eltérések jellemzően purple fringing jelenlétére utalnak, mivel ez a jelenség főként a piros és kék

csatornáknak jelenik meg.

2. Magas kontrasztú területek detektálása

A zöld és piros csatornák különbsége alapján detektálja a nagy kontrasztú részeket (`high_contrast_area`). Erre azért van szükség, mert a purple fringing jellemzően ezek mentén fordul elő.

3. Purple fringing kezdeti maszk létrehozása

A fent kiszámított színekülönbség és kontraszt alapján létrehoz egy bináris maszkot (`purple_fringing`), amely azokat a pixeleket tartalmazza, amelyek mind a színekülönbség, mind a kontraszt szempontjából gyanúsak.

Ez a maszk fekete-fehér képként kerül előállításra (`purple_fringing_image`), ahol a hibás területek fehér színnel jelennek meg.

4. HSV színtér használata a színalapú szűréshez

A bemeneti képet HSV színtérbe konvertálja, amely jobban elkülöníti a színeket és megkönnyíti a lila árnyalatok célzott detektálását.

Három különböző lila árnyalatra definiál színtartományokat, ahol a Hue (H) értékek különböznek a Saturation (S) és a Value (V) egyaránt magas:

- Purple (Hue érték: 260 – 320)
- Blue-purple (Hue érték: 240 – 260)
- Red-purple (Hue érték: 320 – 360)

Ezekből egy színmaszk (`color_mask`) készül, amely egyesíti a különböző tartományokat.

5. Szín és kontraszt együttes figyelembevétele

A `purple_fringing_image` és a `color_mask` kombinálásával megszüri azokat a pixeleket, amelyek egyszerre szín és kontraszt alapján is hibásnak tekinthetők (`combined_mask`).

Ha a kombinált maszkban található aktív pixel, akkor:

6. Kontúr- és területvizsgálat

A függvény kontúrokat keres a maszkon. Ezután kiszámítja az egyes területek méretét.

Ha egy kontúr túl nagy, akkor azt nem tekinti hibának. A kontúrok területének küszöbértékét `area_threshold` változó tartalmazza, amelynek értéke arányosan állítható a kép méretéhez, így skálázható különböző felbontású képekre is. Az elfogadott méretű kontúrokat külön maszkkal (`valid_fringing_mask`) összegyűjti és egy új

maszkot (`contour_mask`) készít. A kontúrok vizsgálatával a függvény segít elkerülni a nagy homogén lila területek téves észlelését.

A függvény a detektálás végén egy igaz vagy hamis érték mellett a hibás területeket tartalmazó bináris maszkot adja vissza, amennyiben nem található purple fringe a képen akkor csak fekete képet ad.

8. kódrészlet: Lila rojtosodás detektálása

```
def purple_fringe_detection(image: np.array):
    b, g, r = cv2.split(image)

    diff_rb = np.abs(r - b)

    contrast_threshold = 30 # Magas kontrasztú területek keresése
    high_contrast_area = np.abs(r - g) > contrast_threshold

    purple_fringing = np.logical_and(diff_rb > 50,
    high_contrast_area)
    purple_fringing_image = np.zeros_like(image)
    purple_fringing_image[purple_fringing] = [255, 255, 255]
    purple_fringing_image = cv2.cvtColor(purple_fringing_image,
    cv2.COLOR_BGR2GRAY)

    hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    lower_purple = np.array([130, 50, 50])
    upper_purple = np.array([160, 255, 255])
    lower_bluepurple = np.array([120, 50, 50])
    upper_bluepurple = np.array([130, 255, 255])
    lower_redpurple = np.array([160, 50, 50])
    upper_redpurple = np.array([180, 255, 255])

    purple_mask = cv2.inRange(hsv_image, lower_purple,
    upper_purple)
    bluepurple_mask = cv2.inRange(hsv_image, lower_bluepurple,
    upper_bluepurple)
    redpurple_mask = cv2.inRange(hsv_image, lower_redpurple,
    upper_redpurple)

    color_mask = cv2.addWeighted(purple_mask, 1, bluepurple_mask,
1, 0)
    color_mask = cv2.addWeighted(color_mask, 1, redpurple_mask, 1,
0)

    combined_mask = cv2.bitwise_and(purple_fringing_image,
    purple_fringing_image, mask = color_mask)
```

```

if np.any(combined_mask != 0):
    contours, _ = cv2.findContours(color_mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    valid_fringing_mask = np.zeros_like(color_mask)
    area_threshold = image.size * 0.0005

    for contour in contours:
        area = cv2.contourArea(contour)
        if area > area_threshold:
            continue
        cv2.drawContours(valid_fringing_mask, [contour], -1,
        255, thickness=cv2.FILLED)

    contour_mask = cv2.bitwise_and(color_mask, color_mask,
    mask=valid_fringing_mask)

    purple_pixels = 0
    if np.any(contour_mask != 0):
        purple_pixels = np.count_nonzero(contour_mask)
        return True, contour_mask, purple_pixels
    else:
        return False, contour_mask, purple_pixels
else:
    purple_pixels = np.count_nonzero(combined_mask)

```

3.8 Képalkotási hibák javítása

A hibák javítását végző függvények a transformations.py fájlban találhatóak meg. A függvények meghívásának sorrendje is nagyon fontos a javítás során, mivel a nem megfelelő sorrend felállításának hiánya befolyásolhatja a javított kép minőségét. A sorrend a következő:

1. Elmosódottság javítása: Egyes javító algoritmusok például kontraszt felerősíthetik az elmosódást, így érdemes először enyhíteni.
2. Expozíciós hibák javítása: Az elmosódás nélküli képek hisztogramja pontosabb, ezért került a második helyre.
3. Purple fringe javítása: A színtorzulások színtérfüggők és érzékenyek a helytelen expozícióra.
4. Vörös szem hiba javítása: Az esetleges színtorzulások és expozíció javítása után érdemes elvégezni, mivel a korrigált, tisztább színekkel rendelkező kép stabilabb eredményt biztosít.

3.8.1 Elmosódottság javítása

Az elmosódott képek javítását a program a `blur_correction` és a hozzá tartozó alfüggvények segítségével végzi. A rendszer az elmosódás típusa alapján egy neurális háló segítségével megbecsüli az adott képhez tartozó Point Spread Function (PSF) kernel alakját, majd a becsült kernel alapján Wiener-dekonvolúcióval visszaállítja az eredeti kép élesebb változatát.

Működése:

A képek javítása két fő lépésből áll:

1. PSF predikció: egy előre betanított neurális hálózat a kép alapján becslést ad az elmosódást okozó kernelre.
2. Wiener dekonvolúció: a becsült kernel segítségével visszafejtjük az elmosódást a képen.

Ez a kombináció lehetővé teszi az elmosódással rendelkező képek automatikus, típusfüggő javítását.

Két különböző PSF prediktor modellt használ a program. Mindkettőt egy-egy elmosódás típus kezelésére. A modellek azonos architektúrával rendelkeznek, de eltérő típusú képeken lettek tanítva.

A hálózat célja, hogy egy bemeneti RGB képből megbecsülje az elmosódást okozó 15×15 -ös PSF kernel értékeit. A bemenetet először 128×128 pixelesre méretezzük. A neurális háló első részét egy egyszerű konvolúciós encoder alkotja, amely két egymás utáni konvolúciós rétegből áll. A első réteg 32 darab szűrőt alkalmaz, a második pedig 64-et, mindkettő után ReLU aktiváció következik, amely a teljes térbeli dimenziót egyetlen értékre tömöríti csatornánként 1×1 méretre, így kivonva a globális jellemzőket.

A kinyert 64 jellemzőt egy teljesen összekapcsolt réteg dolgozza fel, amely 225 kimentet állít elő, ami megfelel a 15×15 méretű kernel összes elemének. Ezután az eredményt egy 4D tenzorral alakítja és ReLU aktivációval biztosítja, hogy a kernel elemei ne legyenek negatívak. Végül normalizálás történik úgy, hogy a kernel elemeinek összege 1 legyen. Ez biztosítja, hogy az eredmény egy valószínűségi eloszlásnak megfelelő súlymátrixként funkcionáljon, amelyet a dekonvolúció során közvetlenül fel lehet használni.

A tanításhoz használt adatforrás DIV2K képgyűjtemény [42] speciális előkészített változatán történt, amely minden elmosódott képhez tartalmazott egy .npy formátumú kernelmátrixot is. A tanítóképek betöltése RGB formátumban történt és egységes pixelméretre méretezve, hogy

illeszkedjenek a modell bementéhez.

A háló tanítása során a cél az volt, hogy a bemeneti képből regresszióval előállítható legyen a hozzá tartozó valós PSF kernelminta. A tanítási célfüggvényként a közönséges négyzetes hiba (MSE) került alkalmazásra, amely a prediktált és a valós kernel közötti eltérést méri. A háló tanítása Adam optimalizálóval történt, $1e-4$ tanulási rátával, 20 epochon keresztül. A tanulás végén a legjobb modell súlyait menti el a program, hogy azok később inferáláskor újra betölthetőek legyenek.

9.kódrészlet: Elmosódott képek javítása

```
def blur_correction(image, blur_type):
    img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    transform = transforms.Compose([
        transforms.Resize((128, 128)),
        transforms.ToTensor()
    ])

    if blur_type == "motion":
        img_tensor =
        transform(Image.fromarray(img_rgb)).unsqueeze(0)
        with torch.no_grad():
            psf = motion_model(img_tensor)[0, 0].numpy()

        img_f = img_rgb.astype(np.float32) / 255.0
        restored = np.zeros_like(img_f)
        for c in range(3):
            restored[:, :, c] =
            wiener_deconvolution(img_f[:, :, c], psf)
    elif blur_type == "defocus":
        img_tensor =
        transform(Image.fromarray(img_rgb)).unsqueeze(0)
        with torch.no_grad():
            psf = defocus_model(img_tensor)[0, 0].numpy()

        img_f = img_rgb.astype(np.float32) / 255.0
        restored = np.zeros_like(img_f)
        for c in range(3):
            restored[:, :, c] =
            wiener_deconvolution(img_f[:, :, c], psf)

    restored_uint8 = (restored * 255).astype(np.uint8)
    restored_img = cv2.cvtColor(restored_uint8, cv2.COLOR_RGB2BGR)

    return restored_img
```

Miután a hálóval sikerült az adott elmosódott képhez tartozó PSF kernel alakját megbecsülni, a kép helyreállítása Wiener-dekonvolúcióval (10. kódrészlet) történik. Ez a módszer frekvenciatartományban működik: először a bemeneti kép minden színcsatornája külön-külön Fourier-transzformációnak van alávetve, ugyanúgy, mint az elmosódást okozó kernelmátrix. A dekonvolúciós lépés lényege, hogy a kernel komplex konjugáltját szorozza a kép spektrumával, majd elosztja a kernel spektrumának abszolút négyzetével, amelyhez egy kis értékű állandót (K) is ad a nevezőhöz a stabilitás érdekében. Ez az állandó (tipikusan 0.01) segít elnyomni a spektrum zajos részeit.

A dekonvolúció után a kapott frekvenciatartománybeli eredményt inverz Fourier-transzformációval alakítja vissza a térbeli tartományba. A végső eredményt klippeli (0 és 1 közé), majd 8 bites formátumba alakítva visszakódolja RGB képpé. Ez a folyamat lehetővé teszi, hogy az eredetihez hasonló, de jóval élesebb képet állítson elő az elmosódott bemenet alapján.

10.kódrészlet: Wiener dekonvolúció

```
def wiener_deconvolution(blurred, kernel, K=0.01):
    h, w = blurred.shape
    kernel = kernel / kernel.sum()
    pad = np.zeros_like(blurred)
    kh, kw = kernel.shape
    pad[:kh, :kw] = kernel

    H = fft2(pad)
    G = fft2(blurred)
    H_conj = np.conj(H)
    F_hat = H_conj * G / (H_conj * H + K)
    result = np.abs(iff2(F_hat))
    return np.clip(result, 0, 1)
```

A fő függvény a folyamat végén visszaadja a javítás során elkészült élesebb képet, amit a felhasználó először a grafikus felületen keresztül tekinthet meg.

3.8.2 Expozíciós hibák javítása

Az alul- és túlexponált képek javítása `exposure_correction` függvény (13. kódrészlet) és a hozzá tartozó alfüggvények használatával történik. Az algoritmus kontrasztkorlátozott adaptív hisztogram kiegyenlítést (CLAHE) és gamma korrekciót kombinál, hogy a kép általános kontrasztját és fényerejét kiegyensúlyozza, miközben a jól exponált részek nincsenek túlerősítve.

Működés:

1. Fényerő becslése:

Az algoritmus először meghatározza a kép átlagos fényerejét a HSV színtér V csatornájának átlaga alapján. Ez a mérték segít meghatározni, hogy a kép erősen-, enyhén alulexponált, normál fényerőjű vagy túlexponált.

11. kódrészlet: A kép átlagos fényerejének meghatározása

```
def get_brightness(image):  
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)  
    return np.mean(hsv[:, :, 2])
```

2. Gamma korrekció

A gamma korrekció segít az általános fényerő korrekciójában. Az algoritmus automatikusan eldönti, hogy alkalmazza-e a gamma korrekciót:

- Erősen alulexponált kép esetén: $\gamma = 1.5$
- Közepesen alulexponált kép esetén: $\gamma = 1.2$
- Túlexponált kép esetén: nem alkalmaz gamma korrekciót

A gamma korrekció csak a V-csatornára kerül alkalmazásra, amely a fényerőt képviseli.

12. kódrészlet: Gamma korrekció alkalmazása a kép világosságának módosítására

```
def apply_gamma_correction(img, gamma):  
    inv_gamma = 1.0 / gamma  
    table = np.array([(i / 255.0) ** inv_gamma * 255 for i in  
                      range(256)]).astype("uint8")  
    return cv2.LUT(img, table)
```

3. CLAHE alkalmazása

Az algoritmus lokálisan javítja a kontrasztot az árnyalat megőrzése mellett. A működés során a teljes V-csatorna több kis csempére oszlik, és mindegyiken külön kontrasztjavítás történik, elkerülve a túlillesztést.

A CLAHE hatásának mértéke a `cliplimit` paraméteren keresztül szabályozható, amelyet a fényerő értéke alapján dinamikusan kerül beállításra. Ha a fényerő értéke alacsonyabb mint 70, akkor erőteljesebb kontrasztjavítást alkalmaz magasabb `cliplimit` értékkel (4.0). Enyhén alulexponált képeknél (70 és 120 közötti fényerő) közepes értéket (3.0) használ. Amennyiben a kép fényereje a túlexponált tartományba esik (160-200), az érték visszafogottabb értékre (2.0) csökken, míg erősen túlexponált képeknél (200 felett) a legkisebb értéket (1.5) alkalmazza, hogy elkerülje a világos területek további túlhangsúlyozását.

4. Színmegőrzés

Annak érdekében, hogy a korrekció után a színek ne tűnjenek mesterségesnek vagy túltelítettnek, az algoritmus a gamma korrekció és a CLAHE által módosított V-csatornát átlagolja. Ez kiegyensúlyozottabb és természetesebb megjelenést biztosít.

A módosított V-csatorna ezután visszakerül az eredeti HSV színtérbe, majd BGR formátumba konvertálva előáll a javított kép.

13. kódrészlet: Automatikusan alkalmazkodó CLAHE-algoritmus színmegőrzéssel

```
def exposure_correction(img, apply_gamma=None, gamma_value = 1.0):
    brightness = get_brightness(img)

    if apply_gamma is None:
        if brightness < 80:
            gamma_value = 1.5
            apply_gamma = True
        elif brightness > 180:
            apply_gamma = False
        else:
            gamma_value = 1.2
            apply_gamma = True

    clip_limit = 2.0
    tile_grid_size = (8, 8)

    if brightness < 70:
        clip_limit = 4.0
    elif brightness < 120:
        clip_limit = 3.0
    elif brightness > 200:
        clip_limit = 1.5
    elif brightness > 160:
        clip_limit = 2.0

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)
    if apply_gamma:
        v = apply_gamma_correction(v, gamma_value)
    clahe = cv2.createCLAHE(clipLimit=clip_limit,
                             tileGridSize=tile_grid_size)
    v_clahe = clahe.apply(v)
    v_final = cv2.addWeighted(v, 0.5, v_clahe, 0.5, 0)
    hsv_clahe = cv2.merge((h, s, v_final))
    result = cv2.cvtColor(hsv_clahe, cv2.COLOR_HSV2BGR)
    return result
```


3.8.3 Piros szem hibák javítása

A piros szem hibákat tartalmazó képek javítása a `red_eye_correction` függvény (14. kódrészlet) segítségével történik. Az algoritmus a bemenetként kapott színes BGR képet és a detektálás során megállapított szemrégiók koordinátáit úgynevezett bounding boxokat használja a vörös effektek korrigálására, amely a következő lépésekben történik:

1. Szemrégió kivágása és színcsatornákra bontása

A szem területe kiemelésre kerül a teljes képből, majd szét lesz bontva kék, zöld és piros csatornákra.

2. Maszk készítése a piros szem detektálásához

A piros szem régiókat olyan pixelekként azonosítja, ahol a piros színcsatorna legalább 20 egységgel meghaladja a kék és zöld csatornák együttes értékét és emellett a piros csatorna értéke önmagában is nagyobb mint 80. Ez a kettő feltétel biztosítja, hogy csak a valóban erőteljesen piros területek kerüljenek kijelölésre, kiszűrve a halványabb vagy természetes vörös tónusokat.

3. Kontúrok kinyerése és maszkolás

A maszk kép kontúrjai kinyerésre kerülnek, majd kiválasztja a legnagyobb területű kontúrt, amely feltételezhetően a piros pupillának felel meg. Ezután morfológiai műveletek zárás és dilatació segítenek a lyukak betömésében, és egy folytonos maszkrégió előállításában.

4. Színkorrekció átlagolt textúrával

A piros szín eltávolításához a kék és zöld csatornák átlaga kerül felhasználásra, amelyet a maszk által kijelölt területeken átmásol a piros pixelek helyére. A cél nem csupán a piros pixelek színének megváltoztatása, hanem a környező szemtextúra és színárnyalat megtartása is. A korrekciót a kép színes formájára alkalmazza, és visszailleszti a kép eredeti szemrégiójába.

14. kódrészlet: Piros szem hiba javítása

```
def Red_eye_correction(img, eyeRects):
    for x,y,w,h in eyeRects:

        eyeImage = img[y:y+h , x:x+w]

        b, g ,r = cv2.split(eyeImage)

        bg = cv2.add(b,g)

        mask  = ( (r>(bg-20)) & (r>80) ).astype(np.uint8)*255

        contours, _ = cv2. findContours(mask.copy() ,
        cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_NONE )
        maxArea = 0
        maxCont = None
        for cont in contours:
            area = cv2.contourArea(cont)
            if area > maxArea:
                maxArea = area
                maxCont = cont
        mask = mask * 0

        cv2.drawContours(mask , [maxCont],0 ,(255),-1 )

        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
        cv2.getStructuringElement(cv2.MORPH_DILATE,(5,5)) )
        mask = cv2.dilate(mask , (3,3) ,iterations=3)
        mean  = (bg / 2).astype(np.uint8)
        mean = mean
        mean = cv2.bitwise_and(mean , mask )
        mean  = cv2.cvtColor(mean ,cv2.COLOR_GRAY2BGR )
        mask = cv2.cvtColor(mask ,cv2.COLOR_GRAY2BGR )
        eye = cv2.bitwise_and(~mask,eyeImage) + mean
        img[y:y+h , x:x+w] = eye

    return img
```

3.8.4 Purple fringe hiba javítása

A lila elszíneződés javítása a purple_fringe_correction függvény (15. kódrészlet) segítségével történik. A feldolgozás egy előzetesen detektált fringing maszk alapján történik, amely jelöli a hibás, lilás árnyalatú régiókat. Az algoritmus első lépésként morfológiai módszerekkel tisztítja a maszkot, majd az elszíneződés súlyosságától függően különböző javítási stratégiákat alkalmaz. A cél a hiba eltüntetése a lehető legkevesebb szín és részletvesztés mellett.

Működés

1. Maszk tisztítása

A javítás előtt a program a maszkot morfológiai műveletekkel tisztítja. Először zárást alkalmaz, amellyel eltünteti a maszkban található apró lyukakat és megszünteti a szakadásokat. Ezt követően nyitási műveletre kerül sor, amely segít eltávolítani a kisebb, izolált zajos foltokat. Ez a tisztítási lépés biztosítja, hogy a kijelölt lila területek összefüggőbbé és pontosabbá váljanak, így a későbbi javítási műveletek megbízhatóbban hajthatók végre.

2. Elszíneződés arányának mérése

A maszkban szereplő fehér pixelek arányából határozza meg a program, hogy a kép mekkora részét érinti a lila elszíneződés. Ez határozza meg, milyen javítási stratégia kerül alkalmazásra.

3. Javítási stratégiák

A lila elszíneződés javítása a hiba mértékéhez igazodik, amelyet a maszkon szereplő hibás pixelek aránya alapján határoz meg. Három eltérő javítási módszert alkalmaz, attól függően, hogy az elszíneződés kis, közepes vagy nagy kiterjedésű.

Ha az érintett területek aránya kisebb, mint öt százalék, akkor precíz, iteratív halványítási technikát alkalmaz a HSV színtérben. Ennek során a maszk által kijelölt pixelek színárnyalatát, telítettségét és fényerejét fokozatosan csökkenti. Az első iterációban 0.95-szörös csökkentést végez, amelyet minden körben tovább mérsékel 0.98-os szorzóval. Minden módosítás után újra lefuttatja a detektálást, és ha a maszk már nem változik, vagy három egymást követő alkalommal nincs változás, a folyamat leáll. Ez a módszer hatékonyan halványítja el a nem kívánt lilás elszíneződést, miközben megtartja a kép színvilágát.

Amennyiben a lila területek aránya öt és húsz százalék közé esik, egy 5x5-ös medián szűrőt alkalmaz a teljes képre. Ezután kizárólag a maszkkal kijelölt régiókat cseréli ki a mediánnal simított változatra. Ez a megközelítés megőrzi a kép többi részének a részletgazdagságát, miközben finoman korrigálja az elszíneződési hibákat.

Ha az elszíneződés aránya meghaladja a húsz százalékot, akkor enyhe Gauss szűrőt alkalmaz a teljes képre, 5x5-ös maszk és nullás szigmaérték mellett. A Gauss szűrés segítségével a program célja, hogy a hiba területén lágyabb átmeneteket hozzon létre, ezzel csökkentve az elszíneződés vizuális hatását anélkül, hogy a kép struktúrája jelentősen sérülne.

15. kódrészlet: Lila rojtosodás javítása

```
def purple_fringe_correction(img, mask):
    initial_reduction_step = 0.95
    reduction_decay = 0.98
    max_iteration = 25

    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    purple_pixels = np.sum(mask > 200)
    purple_ratio = (purple_pixels / img.size) * 100
    iteration = 0
    no_change_iteration = 0
    if purple_ratio < 5:
        prev_mask = mask.copy()
        while np.any(mask) and iteration < max_iteration:
            hsv_img[:, :, 0] = np.where(mask > 0, hsv_img[:, :, 0] *
            initial_reduction_step, hsv_img[:, :, 0])
            hsv_img[:, :, 1] = np.where(mask > 0, hsv_img[:, :, 1]
            * initial_reduction_step, hsv_img[:, :, 1])
            hsv_img[:, :, 2] = np.where(mask > 0, hsv_img[:, :, 2]
            * initial_reduction_step, hsv_img[:, :, 2])

            img = cv2.cvtColor(hsv_img, cv2.COLOR_HSV2BGR)
            _, mask = purple_fringe_detection(img)

            if np.sum(mask != prev_mask) == 0:
                no_change_iteration += 1
                if no_change_iteration == 3:
                    break
            prev_mask = mask.copy()
            initial_reduction_step = max(0.8,
            initial_reduction_step * reduction_decay)
            iteration += 1

            hsv_img = hsv_img.astype(np.uint8)
            result_img = cv2.cvtColor(hsv_img, cv2.COLOR_HSV2BGR)

    elif purple_ratio > 5 and purple_ratio <= 20:
        blurred = cv2.medianBlur(img, 5)
        img[mask > 0] = blurred[mask > 0]
        result_img = img
    elif purple_ratio > 20:
        blurred = cv2.GaussianBlur(img, (5, 5), 0)
        img[mask > 0] = blurred[mask > 0]
        result_img = img
    return result_img
```

3.9 Osztályozás

A program a hibadetektálás és a képi hibák javítását követően lehetőséget biztosít a képek mentésére. A mentés során a felhasználó dönti el, hogy az eredeti vagy a javított képváltozatot szeretné elmenteni. A mentési lépés kézzel indítható gombbal a grafikus felületen keresztül. Ezen felül a mentés automatikus osztályozást is végez a képen látható tartalom alapján egy előre betanított YOLOv8 modell segítségével. A rendszer a képeket kategóriák szerint alkönyvtárakba rendezi, megkönnyítve azok további felhasználását.

3.9.1 Automatikus osztályozás YOLOv8 modellel

A képek osztályozása a `classify` függvényben történik, amely egy betanított YOLOv8 modellt használ. A modell az adott képre vonatkozó legnagyobb valószínűségi osztályt határozza meg, majd a hozzárendelt főkategória szerint alkönyvtárba sorolja a képet.

Az osztályok a programban előre meghatározott osztályok és azok főosztályba csoportosítása alapján működnek. A fő kategóriák a következők: Flowers, Cars, Landscapes, Animals valamint Group of People, melyek a predikció eredményétől függően mappákba rendezik a képeket.

A modell tanítása az `images.cv` [43] oldalon található képekből készült adatbázissal történt. A képek minden osztályhoz külön mappában helyezkedtek el, a YOLOv8 osztályozási tanításhoz szükséges adatstruktúrában. Annotációs fájlokra nem volt szükség, mivel a mappanevek egyértelműen jelölték az osztályokat.

A `data.yaml` fájl tartalmazza az osztályok számát és az `images.cv` alapján létrehozott osztályneveket a `names` listában. A tanítás során biztosítottam, hogy az osztályozási struktúra teljes mértékben megfeleljen a fenti osztályoknak.

A `classify` függvény a prediktált osztály alapján kiválasztja a megfelelő fő kategóriát, és a kép automatikusan a megfelelő alkönyvtárba kerül mentésre.

4 Tesztelés

4.1 Módszer

A program tesztelését nagyfelbontású fényképezőgéppel és mobiltelefonnal készült képekkel végeztem. A valós alkalmazási környezethez való közelség érdekében kifejezetten nagyfelbontású képeket használtam, mivel a mai képalkotó eszközök – különösen a mobiltelefonok és fényképezőgépek – túlnyomórészt ilyen képeket rögzítenek. Ez a választás lehetővé teszi, hogy a program életszerű körülmények között is jól teljesítsen.

A nagyobb felbontás előnyei közé tartozik, hogy a képek több pixelt tartalmaznak, így a részletek – például az élek, textúrák és árnyalatok – jobban elkülönülnek. Ennek köszönhetően a detektáló algoritmusok érzékenyebben reagálnak a finom eltérésekre, és a kontraszt- és élességváltozások is pontosabban mérhetők. Ezzel szemben alacsonyabb felbontás esetén bizonyos hibák elmosódhatnak, így nehezebben észlelhetővé válnak, ami rontja a detektálás és javítás megbízhatóságát. Bár a nagyfelbontású képek feldolgozása hosszabb futási időt és nagyobb memóriaigényt eredményez, ez a kompromisszum elfogadható a tesztelési és fejlesztési szakaszban.

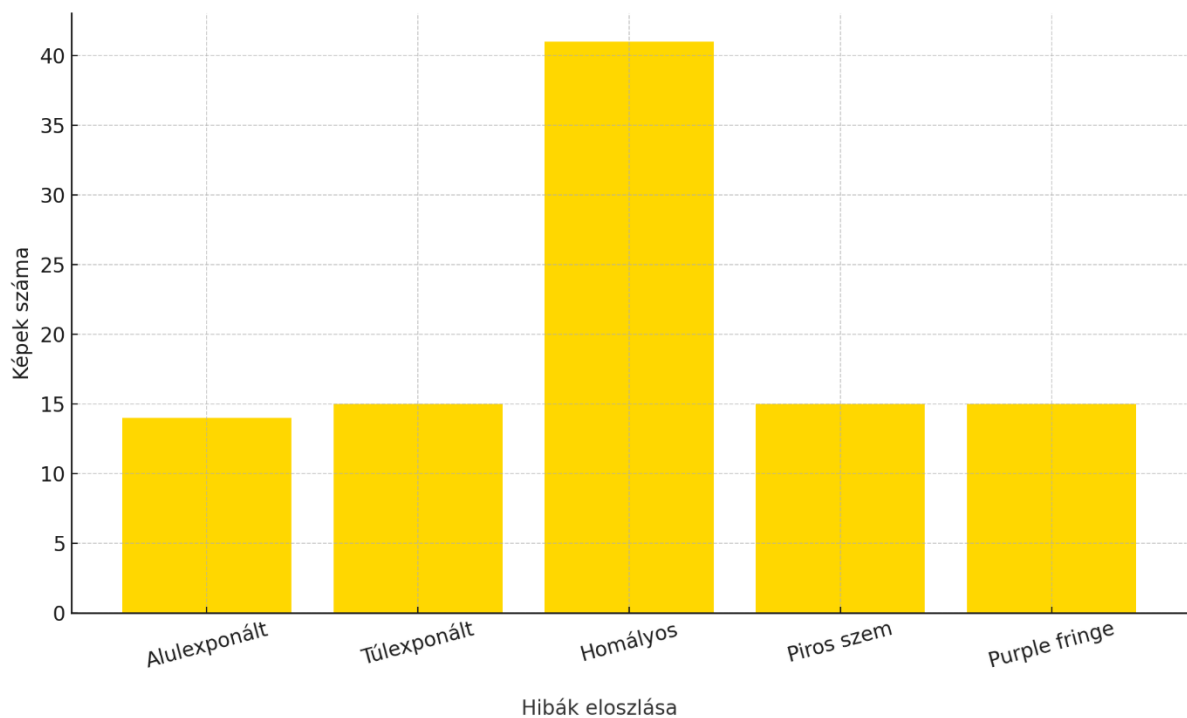
A program minden esetben ugyanazon beállításokkal, változtatás nélküli paraméterekkel futott le, biztosítva ezzel az egységes értékelési környezetet.

4.2 Tesztkészlet

A teszteléshez összeállított képhalmaz összesen 100 képből állt, amelyek különböző típusú képalkotási hibákat reprezentáltak. Ezek közül 40 kép duplikált formában szerepelt – nyolc eredeti kép mindegyike öt példányban –, amelyek az előfeldolgozási lépés tesztelését szolgálták, különösen a hasonlóság-alapú képválogatás (hashing és minőség szerinti kiválasztás) helyességének ellenőrzésére.

Az előfeldolgozás után fennmaradó 68 egyedi kép szolgált a detektálási, javítási és osztályozási funkciók tesztelésére. Ezek hibajellemzői a következőképpen alakultak: 14 kép volt alulexponált, 15 túlexponált, 41 tartalmazott valamilyen homályosságot, 15 képen jelent meg piros szem effektus, míg 15 képen lila rojtosodás volt megfigyelhető (15. ábra).

A képek közül 32 többféle hibát is tartalmazott: 24 képen kétféle hiba (16. és 18. ábra), míg 8 képen három (17. ábra) különböző típusú hiba volt jelen. A tesztkészlet emellett 8 darab előre javított vagy ideálisan rögzített képet is tartalmazott, amelyek nem mutattak képalkotási hibát. Ezek célja a téves pozitív detektálások kizárásának vizsgálata volt.



15. ábra: Különböző hibák eloszlása

A változatos hibajelleg és azok kombinációja lehetővé tette a program valamennyi moduljának – különösen a kombinált hibákat kezelő funkciók – teljes körű és életszerű kiértékelését. A tesztkészletben szereplő, többszörös hibát tartalmazó képek magas aránya (47%) hangsúlyozza, mennyire fontos, hogy a rendszer ne csupán az egyedi hibákat ismerje fel és kezelje, hanem képes legyen azok együttes jelenlétére is megfelelően reagálni.

4.3 Eredmények, levont következtetések

4.3.1 Előfeldolgozás

A tesztelés során az előfeldolgozási lépések – ideértve a képek betöltését, a duplikált képek felismerését, a zajszűrést és a képnevek listázását – összesen mintegy 10 percet vettek igénybe a 100 képből álló adatkészleten. A duplikált képek esetében a program megbízhatóan választotta ki a legjobb minőségű példányokat, a minőségértékelő algoritmus hibakód nélkül futott le. A zajszűrés automatikusan kiválasztott módszerrel zajlott le minden képre. Minden betöltött kép szerepelt a későbbi detektálási, javítási és osztályozási lépésekben is, így az előfeldolgozás sikeresnek tekinthető.

4.3.2 Detektálások

A hibadetektálás képenként átlagosan 1 percet vett igénybe, a pontos idő függött a képmérettől és a hibák jellegétől. Az expozíció értékelése a hisztogram- és pixelintenzitás-eloszlás alapján

történt, amely megbízhatóan különítette el az alul- és túlexponált képeket. A homályosság detektálásánál a Laplace-variancia és a neurális hálózat kombinációja megbízható eredményt adott a homály típusára (defókuszos vagy mozgás) vonatkozóan is.

A piros szem hibák detektálásánál a módszer jól működött egyéni portrékon, azonban csoportképek esetén előfordult, hogy nem az összes hibás terület került felismerésre. A lila rojtosodás detektálásánál a program pontosan azonosította a problémás régiókat, és jól használható maszkkal segítette elő a javítást.

4.3.3 Javítások

A javítási lépések futásideje átlagosan 1–2 perc között mozgott képenként, a hibák számától és kiterjedtségétől függően. Az elmosódás javítása a PSF kernel alapján történt. A Laplace-variancia újraszámítása után egyértelmű volt, hogy a képek élessége nőtt. A defókuszos típusú elmosódás esetén a módszer hatékonyabbnak bizonyult, de mozgási elmosódásnál is javulás volt megfigyelhető.

Az expozíciós hibák korrekciója során a CLAHE algoritmus látványos javulást eredményezett, különösen az alul- és túlexponált képeken. A kiégett (teljesen fehér) vagy túl sötét (teljesen fekete) területek nem javultak, mivel ott nem áll rendelkezésre elegendő vizuális információ. Ez azonban nem tekinthető hibának, hanem a javítási algoritmus természetes korlátja.

A piros szemek javítása nagyrészt sikeres volt, de hasonlóan a detektáláshoz, csoportos jeleneteknél nem minden esetben került javításra az összes hibás szem. A purple fringe javítása során a maszk alapján végzett szintompítás hatékonyan csökkentette a hiba vizuális hatását, miközben a részletek és színek megőrzése is kielégítő volt. A teljes eltüntetés ugyan nem mindenhol történt meg, de ez a hiba természetéből adódik – a kamera optikai rendszerének ismerete nélkül nem várható el tökéletes javítás.

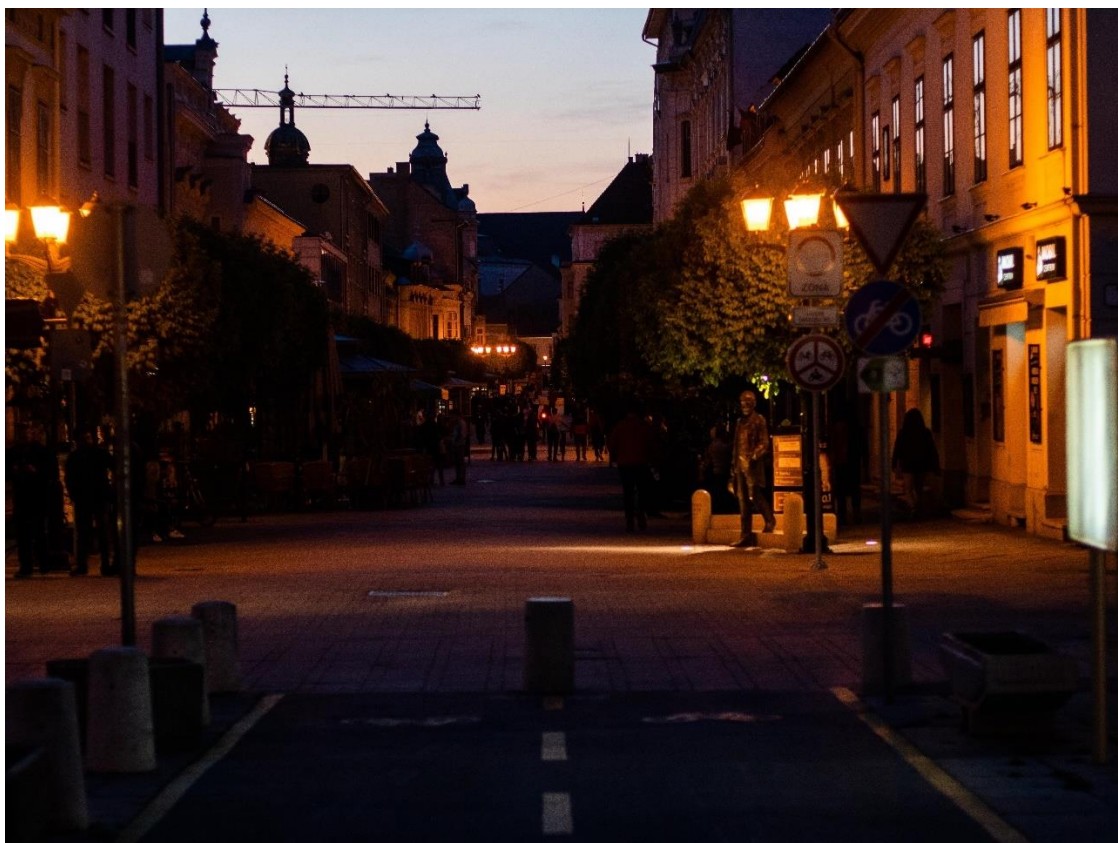
4.3.4 Osztályozás

A képek automatikus osztályozása minden alkalommal lefutott hiba nélkül, a mentés néhány másodpercet vett igénybe képenként. Előfordult, hogy egyes képek nem a leginkább várt mappába kerültek – ez azoknál volt jellemző, amelyek több különböző is tartalmaztak. A modell ilyenkor önállóan döntött a domináns jellemző alapján, és a döntés minden esetben érthetőnek bizonyult. Egyértelműen hibás osztályozás nem fordult elő a tesztelés során.



16. ábra: Tesztkép.

Az eredeti kép (felső) két hibát tartalmaz túlexponálás és elmosódás. Alatta a javított kép.



17. ábra: Tesztkép.
Az eredeti kép (felső) három hibát tartalmaz túlexponálás, elmosódás és lila rojtosodás. Alatta a javított kép.



18. ábra: Tesztkép.
Az eredeti kép (felső) két hibát tartalmaz erőteljes túlexponálás és elmosódás.
Alatta a javított kép.

5 Összegzés

Az elkészült program egy átfogó megoldást nyújt digitális képek automatikus feldolgozására, javítására és strukturált rendszerezésére. A folyamatok révén a felhasználók jobb minőségű képeket kaphatnak, és egy áttekinthető fotóalbumot hozhatnak létre, amelyben a képek könnyebben visszakereshetők. A szoftver működését különféle képfeldolgozó algoritmusok, neurális hálózatok és automatizált javítási eljárások támogatják, amelyek integrált módon járulnak hozzá a magas szintű képminőség eléréséhez.

A tesztelés során a rendszer stabilan és megbízhatóan működött. A képalkotási hibák detektálása pontosan zajlott, és a javítási eljárások is jelentős képminőség-javulást eredményeztek. Az osztályozási funkció a teszthalmoz képeit megfelelő kategóriákba rendezte, és az automatizált mentés is hibamentesen működött.

A továbblépési lehetőségek több irányban is adóttak. A jelenleg alkalmazott küszöbértékek tapasztalati alapon kerültek meghatározásra, ezért ezek automatikus adaptálása gépi tanulás vagy statisztikai optimalizáció segítségével tovább növelheti a detektálás pontosságát. Emellett az osztályozó modell bővítése új kategóriákkal (pl. portré, éjszakai kép, városkép) lehetővé tenné, hogy még komplexebb galériákban is hatékonyan működjön a képek rendezése.

Az előfeldolgozási és javítási lépések esetében érdemes megvizsgálni a több szálon (threading) vagy több folyamaton (multiprocessing) futó megoldásokat, amelyek jelentősen gyorsíthatják a futást, különösen nagy méretű vagy sok képből álló adathalmazok esetében. Ezen kívül a grafikus felület (GUI) további finomhangolása vagy fejlettebb státuszinformációk (például egyéni előrehaladási visszajelzők, részletes logok) bevezetése is növelheti a használhatóságot.

Egy további, a tervezés során megfogalmazott, de ebben a megvalósításban még nem implementált fejlesztési irány a duplikált képek egyesítése. Abban az esetben, ha az azonos jelenetet ábrázoló képek között kisebb eltérések (például zaj, expozíció, elmosódás) tapasztalhatók, a rendszer képes lehetne a legjobb részek automatikus összeillesztésére. Ez a képek összehasonlítása után pixelalapú összeolvasztással vagy több képes HDR-technika alkalmazásával valósulhatna meg. Egy ilyen funkció tovább növelhetné a képminőséget és kiválthatná az emberi döntést a legjobb változat kiválasztásáról.

Összességében az elkészült alkalmazás egy jól működő, rugalmas és bővíthető alapot biztosít a jövőbeli fejlesztésekhez. Lehetőséget ad arra, hogy akár professzionális képkezelő rendszerek irányába is továbbfejlődjön, miközben már most is érezhetően javítja a képek minőségét és strukturált kezelését.

6 Irodalomjegyzék

- [1] Golchubian, A., Marques, O., Nojournian, M.: Photo quality classification using deep learning.
Multimedia Tools and Applications, pp. 22193-22208, 2021.
- [2] Zhang, S., Shen, X., Lin, Z., Měch, R., Costeira J. P., Moura, J. M. F.: Learning to Understand Image Blur.
2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6586-6595, 2018.
- [3] Guo, D., Cheng, Y., Zhou, S., Sim, T.: Correcting over-exposure in photographs.
2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, pp. 515-521, 2010.
- [4] Hou, L., Ji, H., Shen, Z.: Recovering Over-/Underexposed Regions in Photographs.
SIAM Journal on Imaging Sciences, pp. 2213-2235, 2013.
- [5] Su, B., Lu, S., Tan, C. L.: Blurred image region detection and classification.
Proceedings of 19th ACM international conference on Multimedia, pp. 1397-1400, 2011.
- [6] Dai, S., Wu, Y.: Motion from Blur.
2008 IEEE conference on computer vision and pattern recognition, pp. 1-8, 2008.
- [7] Kalalembang, E., Usman, K., Gunawan, I. P.: DCT-based local motion blur detection.
International Conference on Instrumentation, Communication, Information Technology, and Biomedical Engineering, pp. 1-6, 2009.
- [8] Wang, X., Zhang, S., Liang, X., Zhou, H., Zheng, J., Sun, M.: Accurate and Fast Blur Detection Using a Pyramid M-Shaped Deep Neural Network.
IEEE Access, 2019.
- [9] Lee, D.-K., Kim, B.-K., Park, R.-H.: Colourisation in Yxy colour space for purple fringing correction.
IET Image Processing, pp. 891-900, 2012.
- [10] Huang, J., Cui, C., Zhang, C., Shen, Z., Yu, J., Yin, Y.: Learning Multi-Scale Attentive Features for Series Photo Selection
ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 2742-2746, 2020.
- [11] Kuzovkin, D., Pouli, T., Cozot, R., Le Meur, O., Kerverc, J., Bouatouch, K.: Context-aware clustering and assessment of photo collections.
Proceedings of the symposium on Computational Aesthetics, pp. 1-10, 2017.
- [12] Kuzovkin, D., Pouli, T., Cozot, R., Le Meur, O., Kerverc, J., Bouatouch, K.: Image selection in photo albums.
Proceedings of the 2018 ACM on International Conference on Multimedia Retrieval, pp. 397-404, 2018.
- [13] Chang, H., Yu, F., Wang, J., Ashley, D., Finkelstein, A.: Automatic triage for a photo series.

- ACM Transactions on Graphics, pp. 1-10, 2016.
- [14] Bernacki, J.: Automatic exposure algorithms for digital photography. Multimedia Tools and Applications, pp. 12751-12776, 2020.
 - [15] Bansal, R., Raj, G., Choudhury, T.: Blur image detection using Laplacian operator and Open-CV. 2016 International Conference System Modeling & Advancement in Research Trends (SMART), 2016.
 - [16] Bradski, G., Kaehler, A.: Learning OpenCV. O'Reilly Media, Inc, United States of America, p. 571, 2008.
 - [17] Pizer, S. M., Amburn, E. P., Austin, J. D., Cromartie, R., Geselowitz, A., Greer, T., Zuiderveld, K.: Adaptive histogram equalization and its variations. Computer Vision, Graphics, and Image Processing, vol.39, issue 3, pp. 355-368, 1987.
 - [18] Gonzalez, R. C., Woods, R. E.: Digital Image Processing Fourth Edition. Pearson, p. 1019, 2008.
 - [19] Hisztogram minta: <https://barnab.hu/a-histogram-ertelmezese-es-hasznalata/>, letöltés ideje: 2025.04.30.
 - [20] Alpaydin, E.: Introduction to Machine Learning, Fourth Edition. The MIT Press Cambridge, p. 415, 2006.
 - [21] Zhang, Z.: Artificial Neural Network. Multivariate Time Series Analysis in Climate and Environmental Research, pp 1-35, 2018.
 - [22] Zou, J., Han, Y., So, S.-S.: Overview of Artificial Neural Networks. in Artificial Neural Networks, pp. 14-22, 2008.
 - [23] Hadházi, D.: Mély konvolúciós neurális hálózatok. BME IE – 338, p. 48, 2018.
 - [24] Kim, J.-T.: Contrast enhancement using brightness preserving bi-histogram equalization. IEEE Transactions on Consumer Electronics, vol. 43(1), pp. 1-8, 1997.
 - [25] Zuiderveld, K.: Contrast limited adaptive histogram equalization. Graphics gems IV, pp. 474-485, 1994.
 - [26] Levin, A., Fergus, R., Durand, F., Freeman W.: Image and Depth from a Conventional Camera with a Coded Aperture. ACM Transactions on Graphics, vol. 26(3), p. 976, 2007.
 - [27] Zhang, L., Sun, Y., Li, M., Zhang H.: Automated red-eye detection and correction in digital photographs. 2004 International Conference on Image Processing, 2004.
 - [28] Gharbi, M., Chaurasia, G., Paris, S., Durand, F.: Deep Joint Demosaicking and Denoising. ACM Transactions on Graphics, vol. 35(6), pp. 1-12, 2016.
 - [29] Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE, vol. 86(11), pp. 2278-2324, 1998.

- [30] Lutz, M.: Learning Python, 5th Edition.
O'Reilly Media, Inc., p. 1643, 2013.
- [31] Reitz K., Schlusser, T.: The Hitchhiker's Guide to Python
O'Reilly Media, Inc., p. 338, 2016.
- [32] Oliphant, T. E.: Guide to NumPy.
Trelgol Publishing, p. 369, 2006.
- [33] Hunter, J. D.: Matplotlib: A 2D Graphics Environment.
Computing in Science & Engineering, vol. 9(3), pp. 90-95, 2007.
- [34] Clark, J. A.: Pillow Documentation.
Python Imaging Library (PIL) Fork, 2015.
- [35] Krawetz, N.: Perceptual Image Hashes.
Hacker Factor Blog, <https://www.hackerfactor.com/blog/>
- [36] Summerfield, M.: Programming in Python 3: A Complete Introduction to the Python Language.
Addison-Wesley, p. 630, 2010.
- [37] Chollet, F.: Deep Learning with Python.
Manning Publications, p. 384, 2017.
- [38] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T. et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library.
NeurIPS, 2019.
- [39] Visszacsatolt neurális hálózat minta: <https://dataaspirant.com/how-recurrent-neural-network-rnn-works/>, letöltése ideje: 2025.04.30.
- [40] Konvolúciós neurális hálózat minta: <https://medium.com/swlh/neuromorphic-computing-spiking-neural-networks-d8c5755b78e3>, letöltés ideje: 2025.04.30.
- [41] Kaggle Blur Dataset: <https://www.kaggle.com/datasets/kwentar/blur-dataset>, letöltés ideje: 2025.03.12.
- [42] DIV2K Dataset: <https://www.kaggle.com/datasets/takihasan/div2k-dataset-for-super-resolution>, letöltés ideje: 2025.03.15.
- [43] Images.cv Dataset: <https://images.cv>, letöltés ideje: 2025.03.20.
- [44] Haar Cascade szem detektáláshoz: Opencv Github repository.
<https://github.com/opencv/opencv/tree/master/data/haarcascades>, letöltés ideje: 2025.03.15.
- [45] YOLOv8 modell arc detektáláshoz Github.
<https://github.com/akanametov/yolo-face>, letöltés ideje: 2025.03.18.
- [46] Ultralytics (2023). YOLOv8 Documentation.
<https://docs.ultralytics.com>
- [47] McKinney, W.: Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. 2nd ed. O'Reilly Media, 2018.

7 Mellékletek

1. melléklet

Az elkészült teljes szoftverhez tartozó fájlok. Github repository

<https://github.com/Krisiiii98/Fotoalbum-szelektalo-alkalmazas/tree/main>

A program indítása photoselector.py fájl futtatásával lehetséges.

2. melléklet

A szoftverhez tartozó betanított modellek: Google drive link.

<https://drive.google.com/drive/folders/14kDqp9mNPC15qC0vAcdANYRJMtNoTSp?usp=sharing>

A Githubon található program indításához szükségesek.